

HW2 Econ 144

Mason Lee

2025-10-22

```
library(readr)
library(lattice)
library(foreign)
library(MASS)
library(car)
```

```
## Loading required package: carData
```

```
require(stats)
require(stats4)
```

```
## Loading required package: stats4
```

```
library(KernSmooth)
```

```
## KernSmooth 2.23 loaded
## Copyright M. P. Wand 1997-2009
```

```
library(fastICA)
library(cluster)
library(leaps)
library(mgcv)
```

```
## Loading required package: nlme
```

```
## This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

```
library(rpart)
library(pan)
library(mgcv)
library(DAAG)
```

```
##
## Attaching package: 'DAAG'
```

```
## The following object is masked from 'package:car':
##
##      vif
```

```
## The following object is masked from 'package:MASS':  
##  
## hills
```

```
library("TTR")  
library(tis)
```

```
##  
## Attaching package: 'tis'
```

```
## The following object is masked from 'package:TTR':  
##  
## lags
```

```
## The following object is masked from 'package:mgcv':  
##  
## ti
```

```
require("datasets")  
require(graphics)  
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':  
## method from  
## as.zoo.data.frame zoo
```

```
##  
## Attaching package: 'forecast'
```

```
## The following object is masked from 'package:tis':  
##  
## easter
```

```
## The following object is masked from 'package:nlme':  
##  
## getResponse
```

```
require(astsa)
```

```
## Loading required package: astsa
```

```
##  
## Attaching package: 'astsa'
```

```
## The following object is masked from 'package:forecast':  
##  
## gas
```

```
library(RColorBrewer)
library(plotrix)
library(tsibble)
```

```
## Registered S3 method overwritten by 'tsibble':
##   method          from
##   as_tibble.grouped_df dplyr
```

```
##
## Attaching package: 'tsibble'
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, union
```

```
library(ggplot2)
library(tidyr)
library(USgas)
library(fpp3)
```

```
## -- Attaching packages ----- fpp3 1.0.2 --
```

```
## v tibble      3.2.1      v tsibbledata 0.4.1
## v dplyr       1.1.4      v feasts     0.4.2
## v lubridate   1.9.4      v fable      0.4.1
```

```
## -- Conflicts ----- fpp3_conflicts --
```

```
## x feasts::ACF()          masks nlme::ACF()
## x dplyr::between()       masks tis::between()
## x dplyr::collapse()      masks nlme::collapse()
## x lubridate::date()      masks base::date()
## x lubridate::day()       masks tis::day()
## x dplyr::filter()        masks stats::filter()
## x lubridate::hms()       masks tis::hms()
## x tsibble::intersect()   masks base::intersect()
## x lubridate::interval()  masks tsibble::interval()
## x dplyr::lag()           masks stats::lag()
## x lubridate::month()     masks tis::month()
## x lubridate::period()    masks tis::period()
## x lubridate::POSIXct()   masks tis::POSIXct()
## x lubridate::quarter()   masks tis::quarter()
## x dplyr::recode()        masks car::recode()
## x dplyr::select()        masks MASS::select()
## x tsibble::setdiff()     masks base::setdiff()
## x lubridate::today()     masks tis::today()
## x tsibble::union()       masks base::union()
## x lubridate::year()      masks tis::year()
## x lubridate::ymd()       masks tis::ymd()
```

```
data1 <- read_csv("ch4_6-Sheet1.csv")
```

```
## Rows: 152 Columns: 3
## -- Column specification -----
## Delimiter: ","
## dbl (2): actual, forecast
## date (1): date
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(data1)
```

```
## # A tibble: 6 x 3
##   date      actual forecast
##   <date>    <dbl>    <dbl>
## 1 1969-01-01  0.433    0.521
## 2 1969-04-01  1.58     0.373
## 3 1969-07-01  0.290    0.348
## 4 1969-10-01  0.633    0.274
## 5 1970-01-01 -0.470   -0.125
## 6 1970-04-01 -0.157   -0.0500
```

```
data1$aligned <- c(NA, head(data1$forecast, -1))
data1 <- data1[-1, ]
data1$error <- data1$actual - data1$aligned
```

```
head(data1)
```

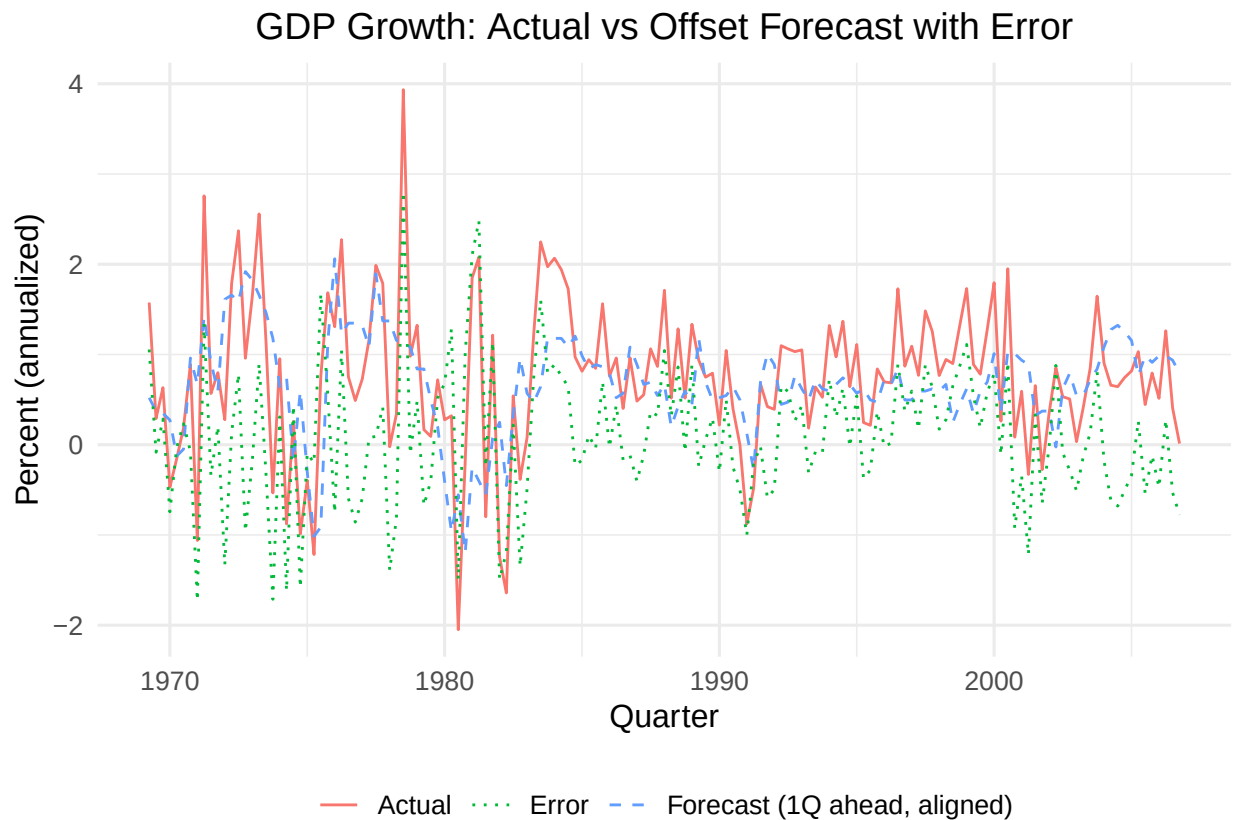
```
## # A tibble: 6 x 5
##   date      actual forecast aligned  error
##   <date>    <dbl>    <dbl>   <dbl>  <dbl>
## 1 1969-04-01  1.58     0.373   0.521   1.06
## 2 1969-07-01  0.290    0.348   0.373  -0.0828
## 3 1969-10-01  0.633    0.274   0.348   0.284
## 4 1970-01-01 -0.470   -0.125   0.274  -0.744
## 5 1970-04-01 -0.157   -0.0500 -0.125  -0.0321
## 6 1970-07-01  0.183    0.961  -0.0500  0.233
```

Testbook A

4.6

```
ggplot(data1, aes(x = date)) +
  geom_line(aes(y = actual, color = "Actual")) +
  geom_line(aes(y = aligned, color = "Forecast (1Q ahead, aligned)",
    linetype = "dashed")) +
  geom_line(aes(y = error, color = "Error"),
    linetype = "dotted") +
```

```
labs(
  title = "GDP Growth: Actual vs Offset Forecast with Error",
  x = "Quarter",
  y = "Percent (annualized)",
  color = NULL
) +
theme_minimal(base_size = 12) +
theme(legend.position = "bottom") +
theme(plot.title = element_text(hjust = 0.5))
```



```
fivenum(data1$actual)
```

```
## [1] -2.0496663  0.3455155  0.7689659  1.1875732  3.9343422
```

```
var(data1$actual)
```

```
## [1] 0.7136576
```

```
mean(data1$actual)
```

```
## [1] 0.7560629
```

```
fivenum(data1$forecast)
```

```
## [1] -1.1962962  0.4839729  0.6682713  0.9974739  2.0604396
```

```
var(data1$forecast)
```

```
## [1] 0.3087317
```

```
mean(data1$forecast)
```

```
## [1] 0.6775047
```

```
fivenum(data1$error)
```

```
## [1] -1.72908180 -0.34463609  0.01718268  0.58510403  2.80366370
```

```
var(data1$error)
```

```
## [1] 0.5937894
```

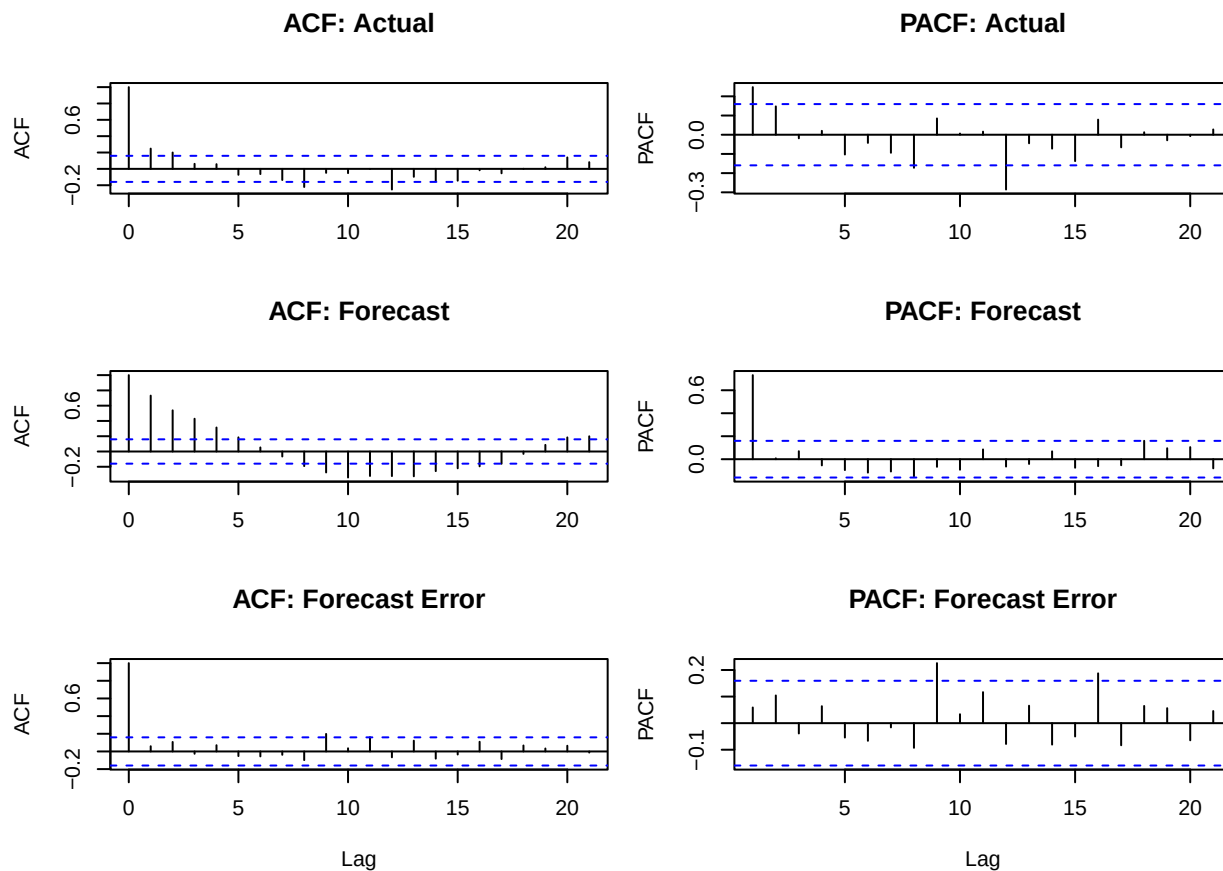
```
mean(data1$error)
```

```
## [1] 0.07855816
```

```
op <- par(mfrow = c(3, 2), mar = c(4, 4, 3, 1))
acf(data1$actual, main = "ACF: Actual", xlab = "", ylab = "ACF")
pacf(data1$actual, main = "PACF: Actual", xlab = "", ylab = "PACF")

acf(data1$forecast, main = "ACF: Forecast", xlab = "", ylab = "ACF")
pacf(data1$forecast, main = "PACF: Forecast", xlab = "", ylab = "PACF")

acf(data1$error, main = "ACF: Forecast Error", xlab = "Lag", ylab = "ACF")
pacf(data1$error, main = "PACF: Forecast Error", xlab = "Lag", ylab = "PACF")
```



```
par(op)
```

In the time series graph, which features the actual percent growth plotted on top of that quarters predicted growth and their difference, we see that growth rates are relatively constant with major variance (backed up by the descriptive statistics). We also see that the error line seems relatively centered at 0, which implies no over/under prediction bias. The actual vs forecast lines are correlated but rarely overlap completely, which means the financial analysts have an idea of how the market will change but their level of certainty is low.

In the ACF/PACF plots, we see that there exists time dependence in the actual and forecasted growth rates, but not in the forecast error. This is exactly what we should expect. For the “actual” ACF/PACF, we see a classic AR(1) or AR(2) plot with strong spikes at lag 1 before falling below the blue line. We also see a strong negative line at lag 12, which suggests some sort of year over year correlation. The Forecast plots also look like they could be AR(1), with a steadily declining ACF and a large early spike in the PACF. Error, on the other hand, is very messy. The ACF has one spike, but the PACF has no clear trend, suggesting noise.

4.8

```
t.test(data1$error, mu = 0)
```

```
##
## One Sample t-test
##
## data: data1$error
```

```
## t = 1.2527, df = 150, p-value = 0.2122
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
## -0.04534827 0.20246460
## sample estimates:
## mean of x
## 0.07855816
```

```
##lags
```

```
e <- data1$error
n <- length(e)
```

```
df1 <- data.frame(
  e_t = e[2:n],
  L1 = e[1:(n-1)]
)
```

```
fit1 <- lm(e_t ~ L1, data = df1)
summary(fit1)
```

```
##
## Call:
## lm(formula = e_t ~ L1, data = df1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.79185 -0.45142 -0.05024  0.48298  2.78504
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.06703    0.06326   1.060   0.291
## L1           0.05952    0.08195   0.726   0.469
##
## Residual standard error: 0.7702 on 148 degrees of freedom
## Multiple R-squared:  0.003552, Adjusted R-squared: -0.003181
## F-statistic: 0.5276 on 1 and 148 DF, p-value: 0.4688
```

```
df2 <- data.frame(
  e_t = e[3:n],
  L1 = e[2:(n-1)],
  L2 = e[1:(n-2)]
)
```

```
fit2 <- lm(e_t ~ L1 + L2, data = df2)
summary(fit2)
```

```
##
## Call:
## lm(formula = e_t ~ L1 + L2, data = df2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```



```
## -1.84975 -0.44867 -0.07074  0.44758  2.93743
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.05937    0.06386   0.930   0.354
## L1           0.05618    0.08259   0.680   0.497
## L2           0.10580    0.08232   1.285   0.201
##
## Residual standard error: 0.7709 on 146 degrees of freedom
## Multiple R-squared:  0.01495,    Adjusted R-squared:  0.001452
## F-statistic: 1.108 on 2 and 146 DF,  p-value: 0.3331
```

```
df3 <- data.frame(
  e_t = e[4:n],
  L1  = e[3:(n-1)],
  L2  = e[2:(n-2)],
  L3  = e[1:(n-3)]
)

fit3 <- lm(e_t ~ L1 + L2 + L3, data = df3)
summary(fit3)
```

```
##
## Call:
## lm(formula = e_t ~ L1 + L2 + L3, data = df3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.85807 -0.44662 -0.07678  0.46706  2.95620
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.06159    0.06471   0.952   0.343
## L1           0.06075    0.08360   0.727   0.469
## L2           0.10642    0.08338   1.276   0.204
## L3          -0.03938    0.08333  -0.473   0.637
##
## Residual standard error: 0.7756 on 144 degrees of freedom
## Multiple R-squared:  0.01613,    Adjusted R-squared:  -0.004367
## F-statistic: 0.787 on 3 and 144 DF,  p-value: 0.503
```

```
fit3r <- lm(e_t ~ 1, data = df3)

anova(fit3, fit3r)
```

```
## Analysis of Variance Table
##
## Model 1: e_t ~ L1 + L2 + L3
## Model 2: e_t ~ 1
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1     144 86.619
## 2     147 88.039 -3    -1.4201 0.787  0.503
```

I did a t-test to test what I visually stated in the prior part ($E(\text{error}) = 0$), and found that we cannot reject our null hypothesis, which claimed that the mean was 0 ($p > 0.05$). There is no statistical bias in the errors. The calculated mean of the error is 0.07, suggesting that forecasters may have under predicted growth during this time frame, but this is not statistically proven. Furthermore, 0 is included in our confidence interval, meaning our true mean being 0 is possible.

Running regression models with up to three lags, we see that none of the lags are statistically relevant for any k levels. The t values are all small, and the p values are all large, suggesting a lack of statistical relevance. The intercepts are close to 0, and the coefficients may be statistically meaningless.

Running an F test on the k=3 model, we again see that we cannot rule out the null hypothesis ($L1 = L2 = L3 = 0$), meaning the lags may have true coefficients of 0. This again proves that there are no biases to be found in the errors.

6.4

a) and b)

```
theta1 <- -2
theta2 <- 1.35
mu <- 0.7
n <- 100

rho_th <- ARMAacf(ma = c(theta1, theta2), lag.max = 10)
round(rho_th, 3)
```

```
##      0      1      2      3      4      5      6      7      8      9     10
## 1.000 -0.689  0.198  0.000  0.000  0.000  0.000  0.000  0.000  0.000  0.000
```

```
set.seed(10000)

y_sim <- as.numeric(mu + arima.sim(model = list(ma = c(theta1, theta2)), n = n))

acf_y <- acf(y_sim, lag.max = 10, plot = FALSE, demean = TRUE)$acf
acf_y
```

```
## , , 1
##
##      [,1]
## [1,] 1.000000000
## [2,] -0.713810638
## [3,] 0.234616934
## [4,] -0.005525419
## [5,] -0.013525045
## [6,] -0.009979723
## [7,] 0.025068671
## [8,] -0.005797255
## [9,] -0.064585666
## [10,] 0.151432081
## [11,] -0.150965956
```

Comparing the theoretical lag vs the simulated lag, we see that they are very similar. ARMAacf from the base r stats package does a great job simulating the data, and then doing a 10 lag acf based on simulated numbers matches up with the early two lags. They aren't identical, but they are close enough to say that our theoretical acf is correct. After the first two lags, the theoretical acf is all 0. This isn't the case for our simulated data, but none of the lags after lag 2 are super significant (highest is 0.151 at lag 10).

6.5

a)

```
fit_ma2 <- arima(y_sim, order = c(0, 0, 2), include.mean = TRUE, method = "ML")
fit_ma2
```

```
##
## Call:
## arima(x = y_sim, order = c(0, 0, 2), include.mean = TRUE, method = "ML")
##
## Coefficients:
##          ma1          ma2  intercept
##        -1.5006  0.6992      0.7472
## s.e.    0.0690  0.0698      0.0281
##
## sigma^2 estimated as 1.971:  log likelihood = -177.26,  aic = 362.52
```

```
ARMAacf(ma = c(-1.5006, 0.6992), lag.max = 10)[c(2, 3)]
```

```
##          1          2
## -0.6816458  0.1869178
```

This MA(2) model has different data generating figures than our original 6.4 formula, but the implied autocorrelations are similar, when calculated out. The function is non-invertible, but attempting to invert anyways leads to similar results.

b)

```
mu      <- 0.7
theta1  <- -2
theta2  <- 1.35
eps_t   <- 0.4
eps_tm1 <- -1.2

f1 <- mu + theta1*eps_t + theta2*eps_tm1
f2 <- mu + theta2*eps_t
f3 <- mu

f1
```

```
## [1] -1.72
```

```
f2
```

```
## [1] 1.24
```

```
f3
```

```
## [1] 0.7
```

6.6

```
thetaA <- 0.8  
thetaB <- 1.25
```

```
m <- 8
```

```
op1 <- (-thetaA)^(1:m)  
op2 <- (-thetaB)^(1:m)
```

```
round(op1, 4)
```

```
## [1] -0.8000  0.6400 -0.5120  0.4096 -0.3277  0.2621 -0.2097  0.1678
```

```
round(op2, 4)
```

```
## [1] -1.2500  1.5625 -1.9531  2.4414 -3.0518  3.8147 -4.7684  5.9605
```

Based on the acf coefficients, we would prefer to use option 1 (op1) for forecasting because it is invertable. Its coefficients trend towards 0. Option 2, on the other hand, is not invertable and therefore does not have asymptotic behavior, making it ineffective in forecasting.

6.10

```
apl <- read_csv("HistoricalQuotes.csv")
```

```
## Rows: 2518 Columns: 6  
## -- Column specification -----  
## Delimiter: ","  
## chr (5): Date, Close/Last, Open, High, Low  
## dbl (1): Volume  
##  
## i Use 'spec()' to retrieve the full column specification for this data.  
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

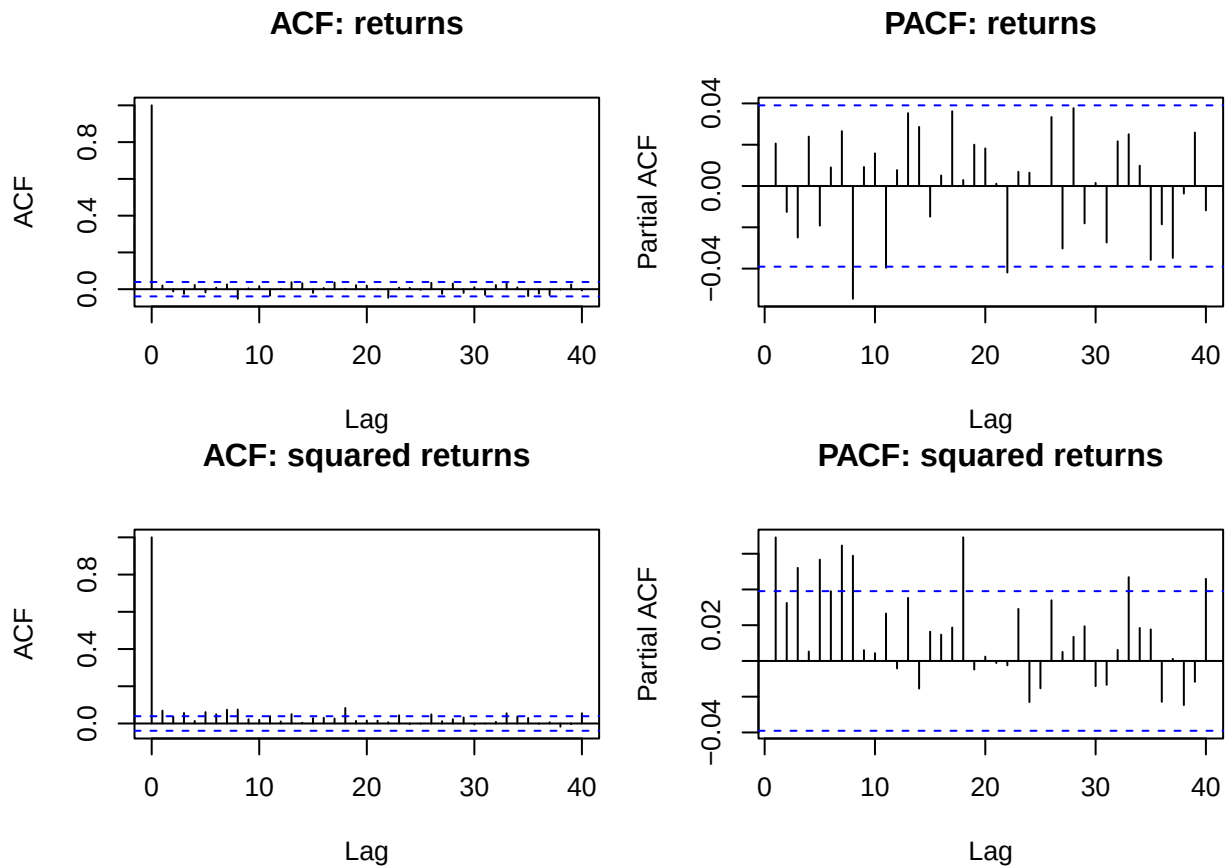
```
head(apl)
```

```
## # A tibble: 6 x 6
##   Date       'Close/Last'      Volume Open    High    Low
##   <chr>      <chr>          <dbl> <chr>   <chr>   <chr>
## 1 02/28/2020 $273.36      106721200 $257.26 $278.41 $256.37
## 2 02/27/2020 $273.52       80151380 $281.1  $286    $272.96
## 3 02/26/2020 $292.65      49678430 $286.53 $297.88 $286.5
## 4 02/25/2020 $288.08      57668360 $300.95 $302.53 $286.13
## 5 02/24/2020 $298.18      55548830 $297.26 $304.18 $289.23
## 6 02/21/2020 $313.05      32426420 $318.62 $320.45 $310.5
```

```
library(dplyr)
apl2 <- apl |>
  mutate(
    Date = as.Date(Date, format = "%m/%d/%Y"),
    Close = as.numeric(gsub("$,", "", `Close/Last`))
  ) |>
  arrange(Date)

r <- diff(log(apl2$Close))
r_dates <- apl2$Date[-1]

par(mfrow = c(2,2), mar = c(4,4,3,1))
acf(r, lag.max = 40, main = "ACF: returns", xlab = "Lag")
pacf(r, lag.max = 40, main = "PACF: returns", xlab = "Lag")
acf(r^2, lag.max = 40, main = "ACF: squared returns", xlab = "Lag")
pacf(r^2, lag.max = 40, main = "PACF: squared returns", xlab = "Lag")
```



```
par(mfrow = c(1,1))
```

```
fit_mean <- auto.arima(r, seasonal = FALSE, stepwise = FALSE, approximation = FALSE)
fit_mean
```

```
## Series: r
## ARIMA(2,0,3) with non-zero mean
##
## Coefficients:
##          ar1      ar2      ma1      ma2      ma3      mean
##        -1.1985  -0.7873   1.2240   0.8133  -0.0144   9e-04
## s.e.    0.2438    0.2590   0.2461   0.2779    0.0356   3e-04
##
## sigma^2 = 0.0002647:  log likelihood = 6797.91
## AIC=-13581.81   AICc=-13581.77   BIC=-13541
```

```
fc_5 <- forecast(fit_mean, h = 5)
fc_10 <- forecast(fit_mean, h = 10)
fc_20 <- forecast(fit_mean, h = 20)
```

```
fc_5
```

```
##      Point Forecast      Lo 80      Hi 80      Lo 95      Hi 95
## 2518  -6.077205e-05 -0.02090916  0.02078762 -0.03194563  0.03182409
```

```
## 2519 4.175876e-03 -0.01667928 0.02503104 -0.02771934 0.03607109
## 2520 -2.357512e-03 -0.02321289 0.01849787 -0.03425306 0.02953804
## 2521 2.137843e-03 -0.01872631 0.02300200 -0.02977112 0.03404681
## 2522 1.894053e-03 -0.01898544 0.02277354 -0.03003837 0.03382648
```

```
fc_10
```

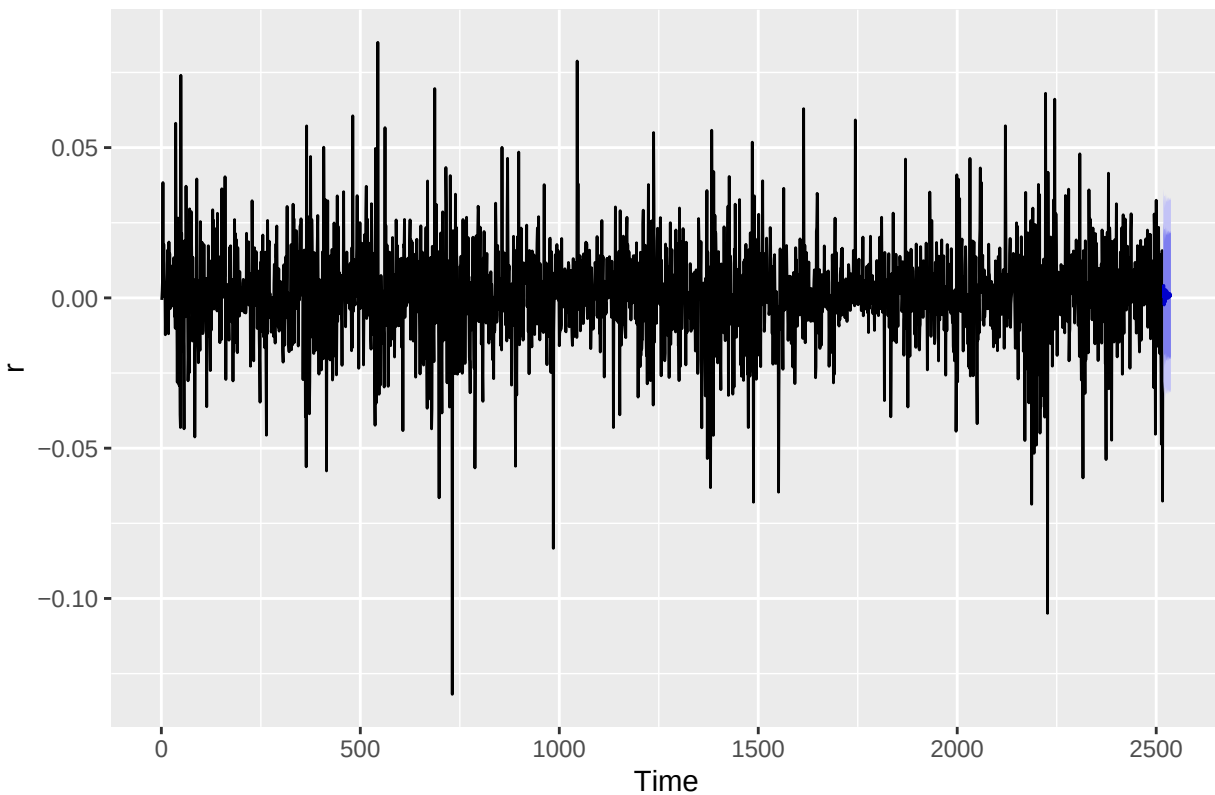
```
##      Point Forecast      Lo 80      Hi 80      Lo 95      Hi 95
## 2518 -6.077205e-05 -0.02090916 0.02078762 -0.03194563 0.03182409
## 2519 4.175876e-03 -0.01667928 0.02503104 -0.02771934 0.03607109
## 2520 -2.357512e-03 -0.02321289 0.01849787 -0.03425306 0.02953804
## 2521 2.137843e-03 -0.01872631 0.02300200 -0.02977112 0.03404681
## 2522 1.894053e-03 -0.01898544 0.02277354 -0.03003837 0.03382648
## 2523 -1.353048e-03 -0.02223812 0.01953202 -0.03329401 0.03058791
## 2524 2.730524e-03 -0.01815461 0.02361566 -0.02921053 0.03467158
## 2525 3.928943e-04 -0.02049691 0.02128270 -0.03155531 0.03234109
## 2526 -2.054777e-05 -0.02091607 0.02087498 -0.03197750 0.03193640
## 2527 2.315423e-03 -0.01858146 0.02321231 -0.02964360 0.03427445
```

```
fc_20
```

```
##      Point Forecast      Lo 80      Hi 80      Lo 95      Hi 95
## 2518 -6.077205e-05 -0.02090916 0.02078762 -0.03194563 0.03182409
## 2519 4.175876e-03 -0.01667928 0.02503104 -0.02771934 0.03607109
## 2520 -2.357512e-03 -0.02321289 0.01849787 -0.03425306 0.02953804
## 2521 2.137843e-03 -0.01872631 0.02300200 -0.02977112 0.03404681
## 2522 1.894053e-03 -0.01898544 0.02277354 -0.03003837 0.03382648
## 2523 -1.353048e-03 -0.02223812 0.01953202 -0.03329401 0.03058791
## 2524 2.730524e-03 -0.01815461 0.02361566 -0.02921053 0.03467158
## 2525 3.928943e-04 -0.02049691 0.02128270 -0.03155531 0.03234109
## 2526 -2.054777e-05 -0.02091607 0.02087498 -0.03197750 0.03193640
## 2527 2.315423e-03 -0.01858146 0.02321231 -0.02964360 0.03427445
## 2528 -1.587129e-04 -0.02105583 0.02073841 -0.03211810 0.03180067
## 2529 9.673680e-04 -0.01993200 0.02186674 -0.03099546 0.03293020
## 2530 1.565703e-03 -0.01933567 0.02246708 -0.03040019 0.03353159
## 2531 -3.798375e-05 -0.02093962 0.02086365 -0.03200428 0.03192831
## 2532 1.412944e-03 -0.01948894 0.02231483 -0.03055373 0.03337962
## 2533 9.366298e-04 -0.01996626 0.02183952 -0.03103158 0.03290484
## 2534 3.651454e-04 -0.02053840 0.02126869 -0.03160406 0.03233435
## 2535 1.425078e-03 -0.01947850 0.02232865 -0.03054418 0.03339433
## 2536 6.046961e-04 -0.02029905 0.02150845 -0.03136483 0.03257422
## 2537 7.534129e-04 -0.02015075 0.02165758 -0.03121675 0.03272358
```

```
autoplot(fc_20) +
  labs(title = "Apple log-returns: ARIMA mean model forecast (h=20)")
```

Apple log-returns: ARIMA mean model forecast (h=20)



```
last_price <- tail(apl2$Close, 1)
mu_hat <- as.numeric(fc_20$mean)

logP0 <- log(last_price)
logP_path <- logP0 + cumsum(mu_hat)
price_path <- exp(logP_path)

data.frame(
  Date = seq(from = tail(r_dates, 1) + 1, by = "days", length.out = length(price_path)),
  PriceForecast = price_path
) |>
head(10)
```

```
##      Date PriceForecast
## 1 2020-02-29      273.3434
## 2 2020-03-01      274.4872
## 3 2020-03-02      273.8409
## 4 2020-03-03      274.4269
## 5 2020-03-04      274.9472
## 6 2020-03-05      274.5754
## 7 2020-03-06      275.3262
## 8 2020-03-07      275.4344
## 9 2020-03-08      275.4287
## 10 2020-03-09      276.0672
```

For this question, I used Apple's daily prices from March 2010 to February 2020. An ARIMA(2,0,3) with a

very small non-zero mean fit the mean dynamics best (though a simpler low-order ARMA would give similar mean forecasts). The ACF of returns shows low day-to-day correlation and the PACF has no consistent spikes—returns are close to white noise. In contrast, the ACF/PACF of squared returns display clear persistence, indicating volatility clustering. Thus, volatility is predictable, but the direction (mean) of returns is not.

Textbook B

6.2

```
library(fpp2)

## -- Attaching packages ----- fpp2 2.5 --

## v fma      2.5      v expsmooth 2.3

## -- Conflicts ----- fpp2_conflicts --
## x forecast::easter()      masks tis::easter()
## x forecast::getResponse() masks nlme::getResponse()

##
## Attaching package: 'fpp2'

## The following object is masked from 'package:fpp3':
##
##      insurance

## The following object is masked from 'package:astsa':
##
##      oil

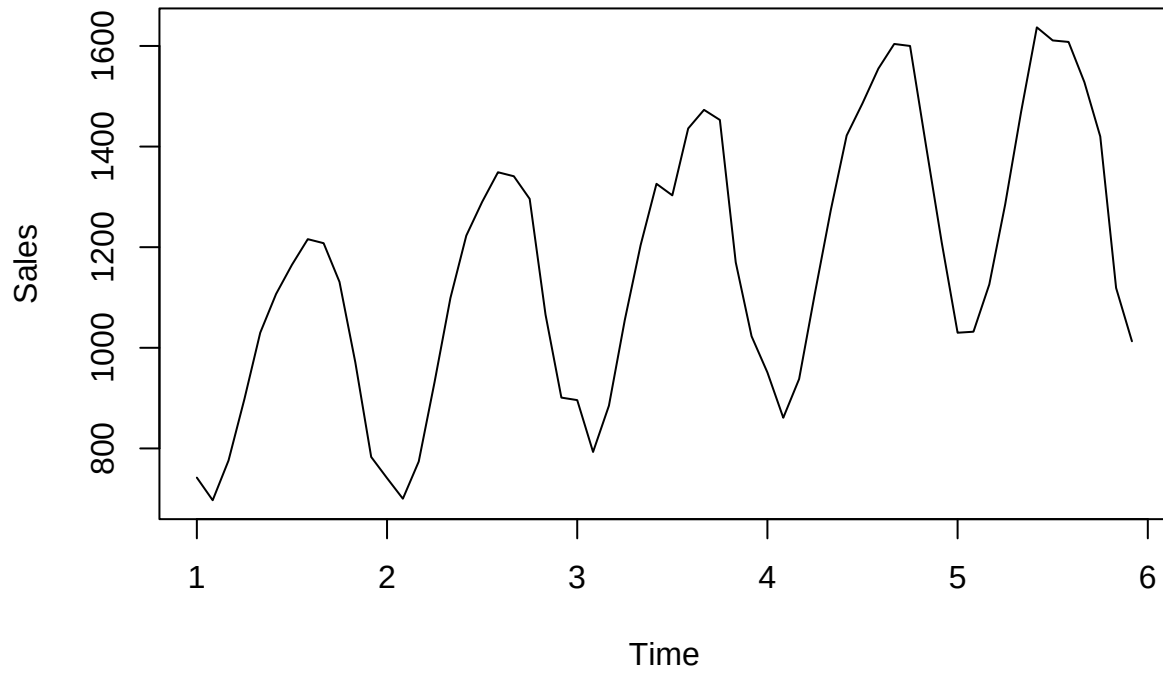
data <- plastics
```

a)

```
y <- as.vector(t(as.matrix(data)))
y_ts <- ts(y, start = c(1, 1), frequency = 12)

plot(y_ts, main = "Plastics: monthly sales (thousands)", ylab = "Sales", xlab = "Time")
```

Plastics: monthly sales (thousands)

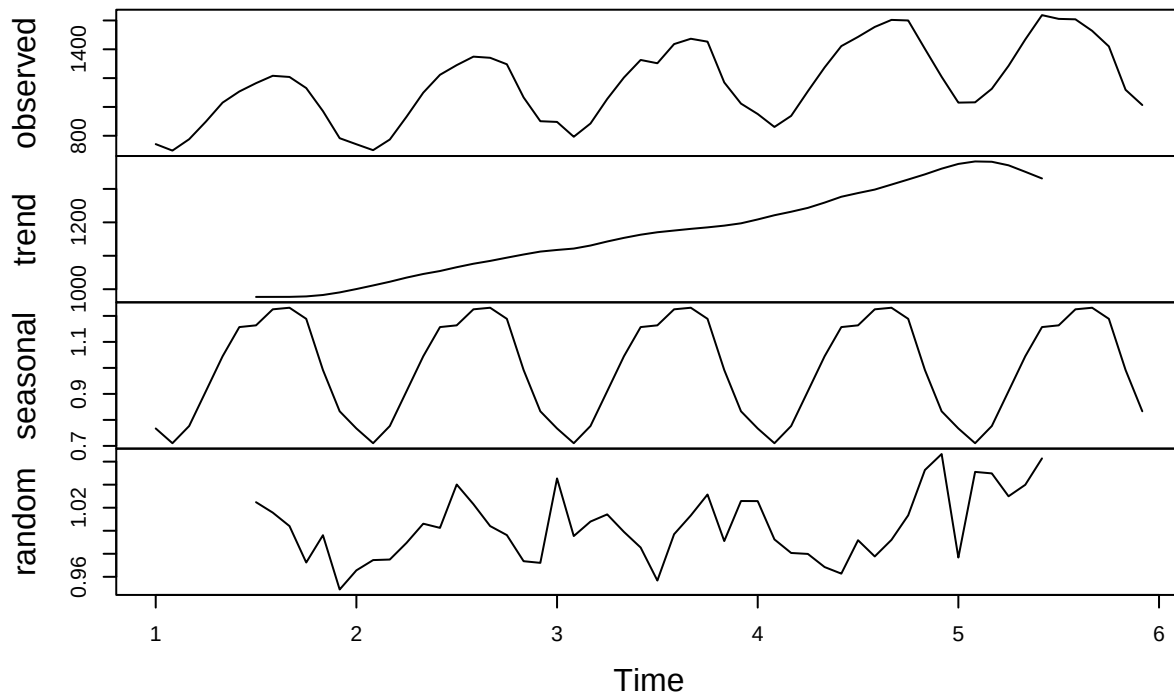


There is a clear trend in the plastics data. Every year, the monthly sales bottom out in January before peaking in summer (June/July).

b)

```
dc <- decompose(y_ts, type = "multiplicative")
plot(dc)
```

Decomposition of multiplicative time series



c)

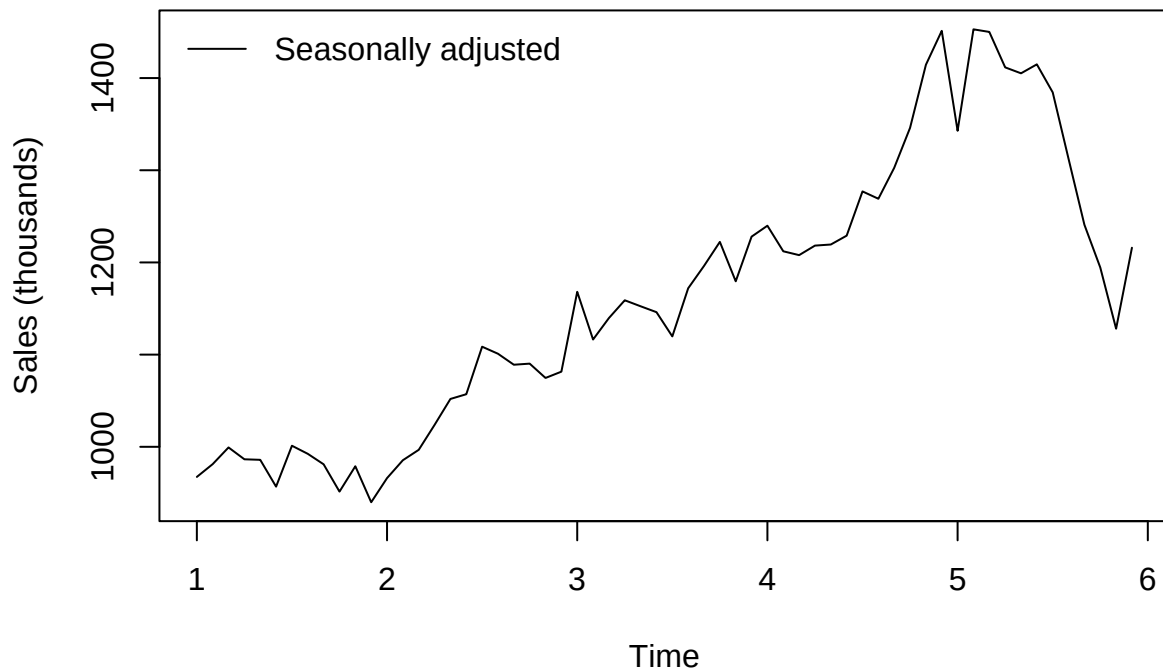
Yes, this supports the time series plot in part a. There is a clear upwards trend and seasonal pattern in this data.

d)

```
sa_ts <- y_ts / dc$seasonal

ts.plot(cbind(SeasonallyAdjusted = sa_ts),
        ylab = "Sales (thousands)",
        main = "Plastics: Seasonally Adjusted Sales (multiplicative)",
        lty = c(1, 2))
legend("topleft", c("Seasonally adjusted"),
       lty = c(1, 2), bty = "n")
```

Plastics: Seasonally Adjusted Sales (multiplicative)



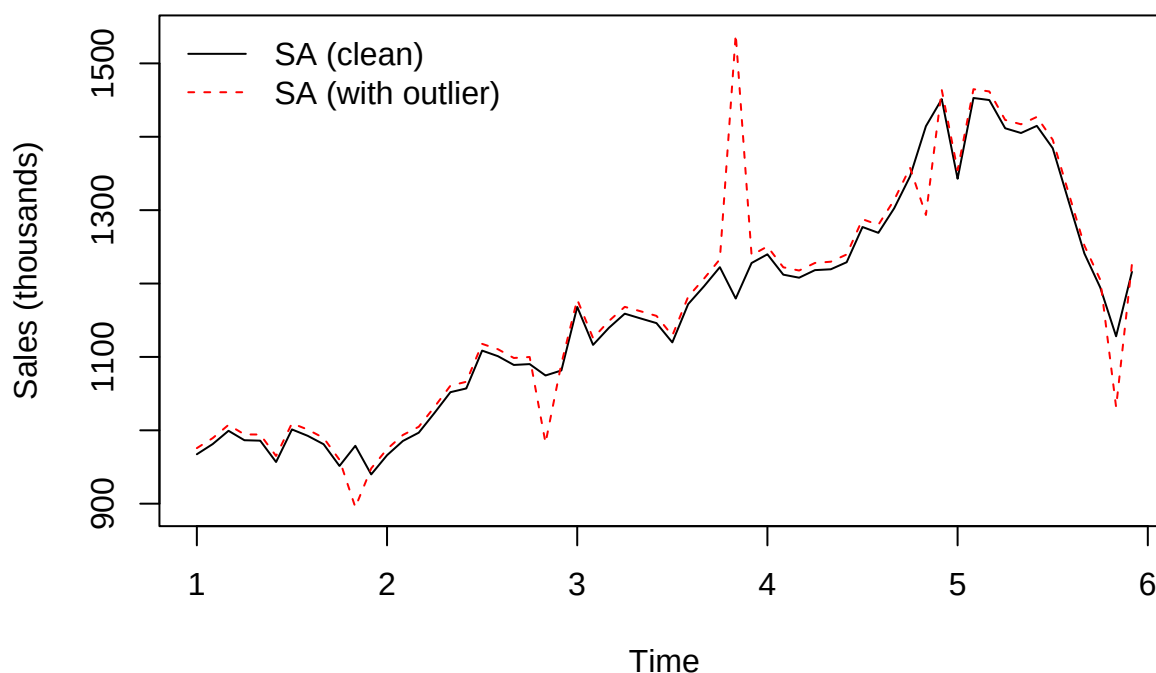
Adjusting seasonally, we see that the graph is much more linear before it falls off due to a lack of 6th year data. The upwards trend is apparent and matches what we see in the main time series plot, where each year is more profitable than the last despite a clear seasonal pattern.

```
y_1 <- y_ts
y_1[35] <- y_1[35] + 500

dc_1 <- decompose(y_1, type = "multiplicative")
sa_1 <- y_1 / dc_1$seasonal

ts.plot(cbind(Original_SA = sa_ts, Outlier_SA = sa_1),
        lty = c(1, 2), col = c("black", "red"),
        ylab = "Sales (thousands)",
        main = "Seasonally Adjusted Series: Effect of a Single Outlier")
legend("topleft", c("SA (clean)", "SA (with outlier)"),
        lty = c(1, 2), col = c("black", "red"), bty = "n")
```

Seasonally Adjusted Series: Effect of a Single Outlier



Adding in an outlier keeps most of the plot consistent, but adds in a major spike at the outlier and a lower correction 12 months away (both directions). This is what we would expect when changing one point in a seasonal trend graph.

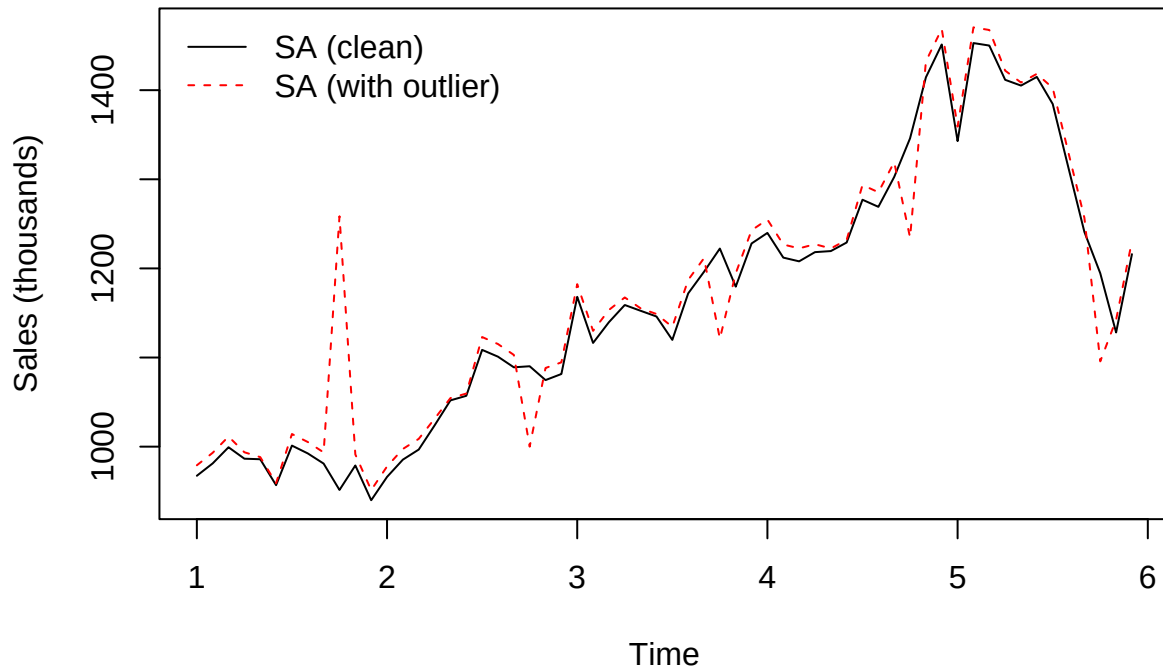
f)

```
y_2 <- y_ts
y_2[10] <- y_2[10] + 500

dc_2 <- decompose(y_2, type = "multiplicative")
sa_2 <- y_2 / dc_2$seasonal

ts.plot(cbind(Original_SA = sa_ts, Outlier_SA = sa_2),
       lty = c(1, 2), col = c("black", "red"),
       ylab = "Sales (thousands)",
       main = "Seasonally Adjusted Series: Effect of a Single Outlier")
legend("topleft", c("SA (clean)", "SA (with outlier)"),
      lty = c(1, 2), col = c("black", "red"), bty = "n")
```

Seasonally Adjusted Series: Effect of a Single Outlier

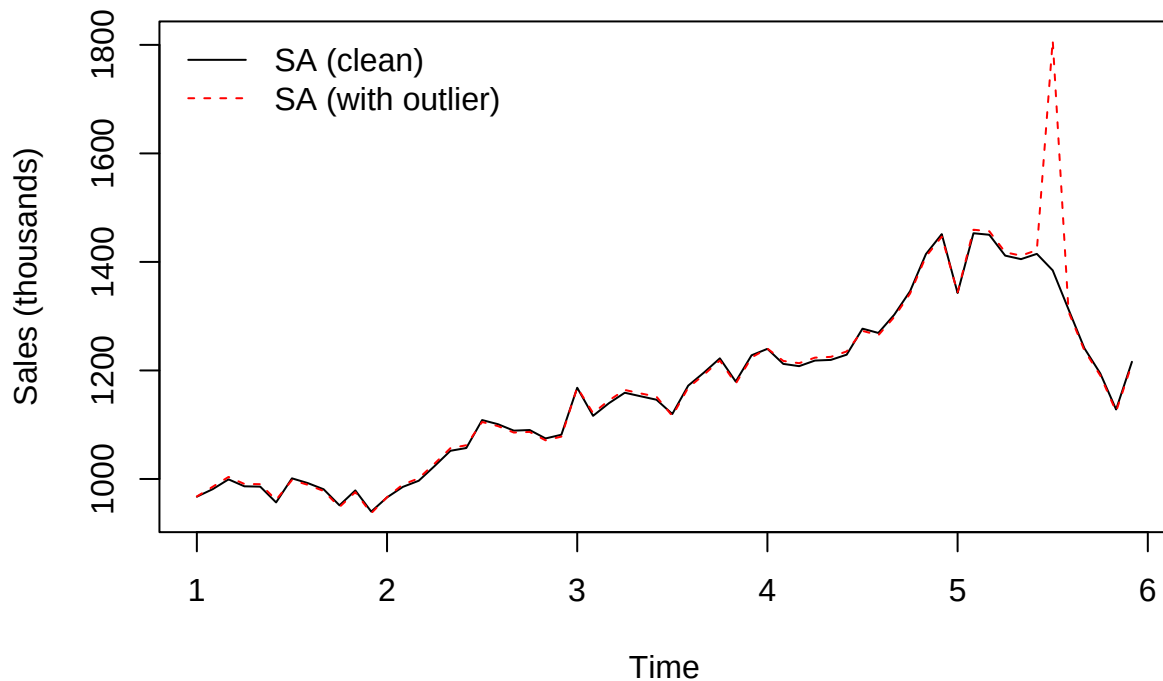


```
y_3 <- y_ts
y_3[55] <- y_2[55] + 500

dc_3 <- decompose(y_3, type = "multiplicative")
sa_3 <- y_3 / dc_3$seasonal

ts.plot(cbind(Original_SA = sa_ts, Outlier_SA = sa_3),
        lty = c(1, 2), col = c("black", "red"),
        ylab = "Sales (thousands)",
        main = "Seasonally Adjusted Series: Effect of a Single Outlier")
legend("topleft", c("SA (clean)", "SA (with outlier)"),
       lty = c(1, 2), col = c("black", "red"), bty = "n")
```

Seasonally Adjusted Series: Effect of a Single Outlier



When the outlier is at the beginning, we see a similar pattern to when it is in the middle. There is a large spike followed by seasonal corrections down the line

When the outlier is at the end, there is no change in the graph until the spike at the outlier.

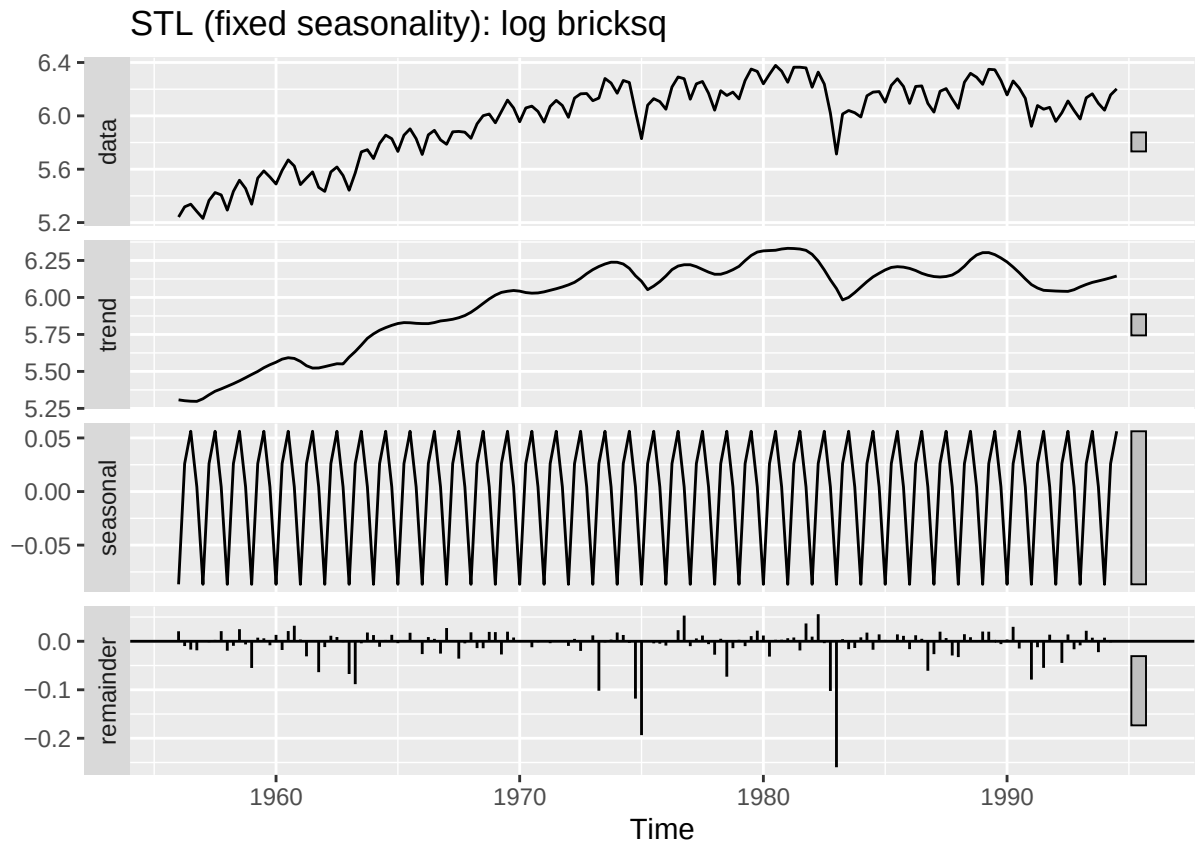
6.6

```
y <- bricksq
```

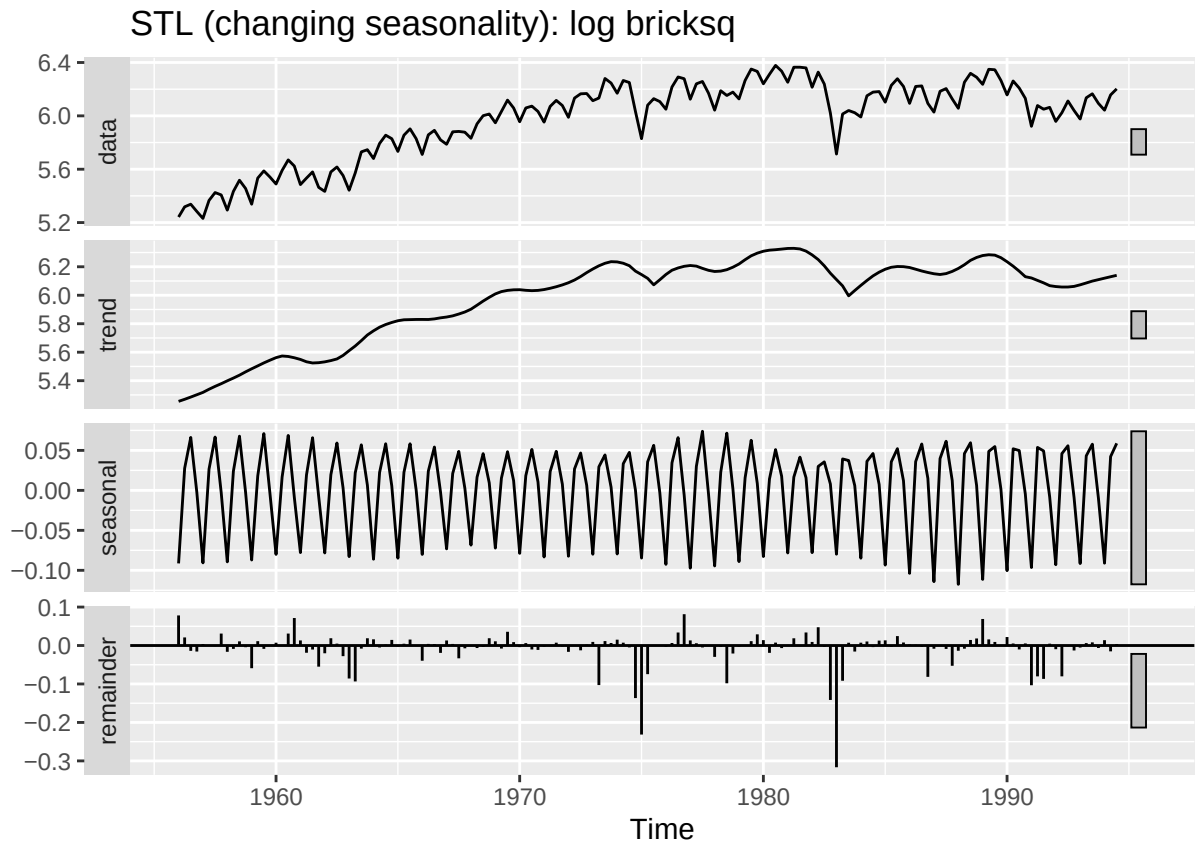
a)

```
yl <- log(y)

stl_fixed <- stl(yl, s.window = "periodic", robust = TRUE)
autoplot(stl_fixed) + labs(title = "STL (fixed seasonality): log bricksq")
```



```
stl_chg <- stl(y1, s.window = 7, robust = TRUE)
autoplot(stl_chg) + ggplot2::labs(title = "STL (changing seasonality): log bricksq")
```

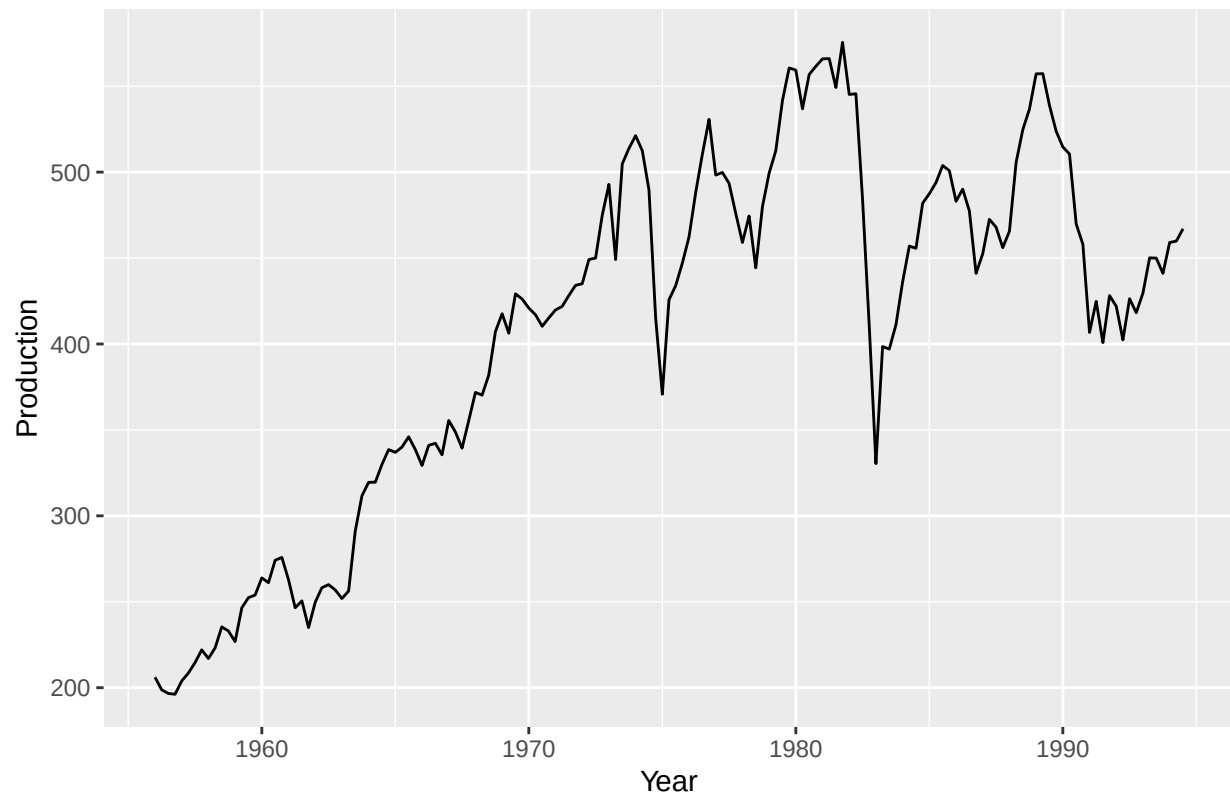
Whether you hold seasonality constant or change it, the stl plots are very similar. General shapes are the same, but the stl with changing seasons seems a little smoother than fixed seasonality. The remainders are also slightly less dramatic, with most of them going towards 0 even though the large spikes remain mostly the same.

b)

```
sa_fixed_log <- seasadj(stl_fixed)
sa_fixed     <- exp(sa_fixed_log)

autoplot(sa_fixed) +
  labs(title = "Bricks (seasonally adjusted) - fixed seasonality",
       x = "Year", y = "Production")
```

Bricks (seasonally adjusted) – fixed seasonality

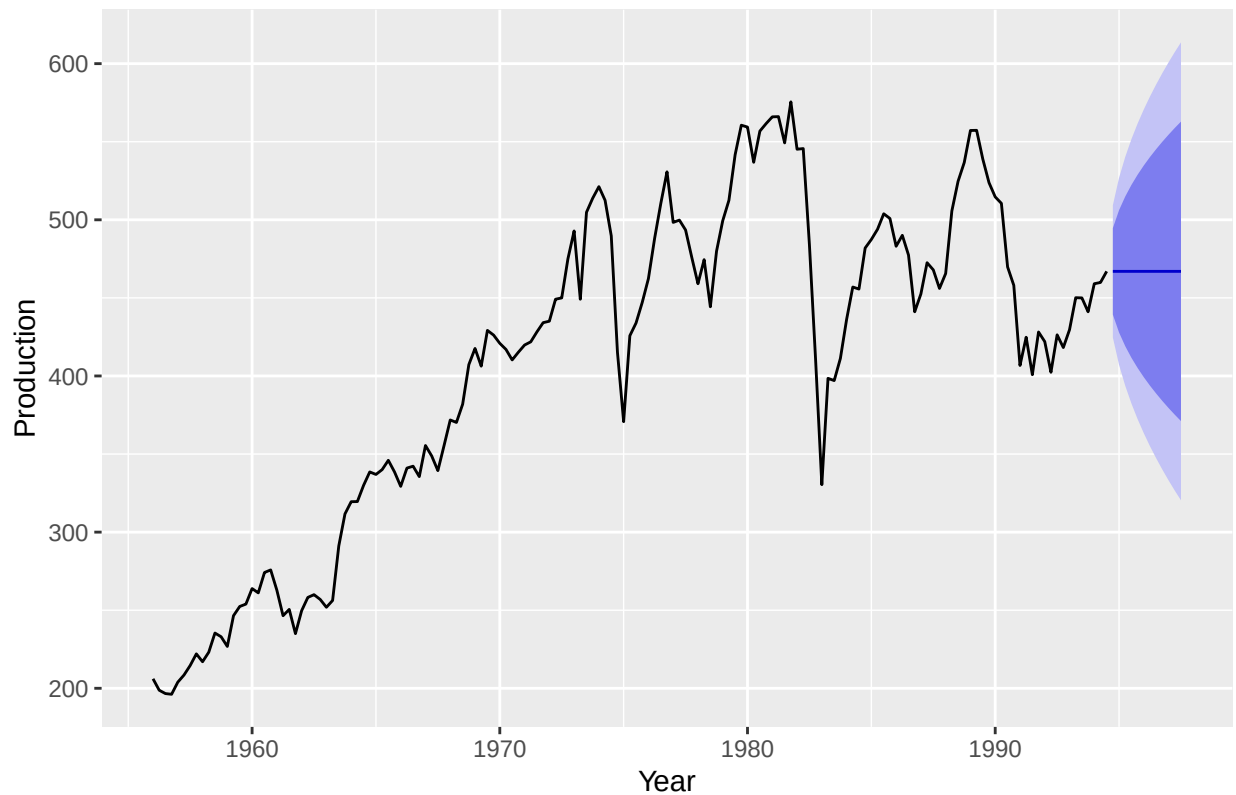


c)

```
h <- 12
fc_fixed <- forecast::naive(sa_fixed, h = h)

autoplot(fc_fixed) +
  labs(title = "Naïve forecasts of seasonally adjusted bricks (fixed seasonality)",
        x = "Year", y = "Production")
```

Naïve forecasts of seasonally adjusted bricks (fixed seasonality)



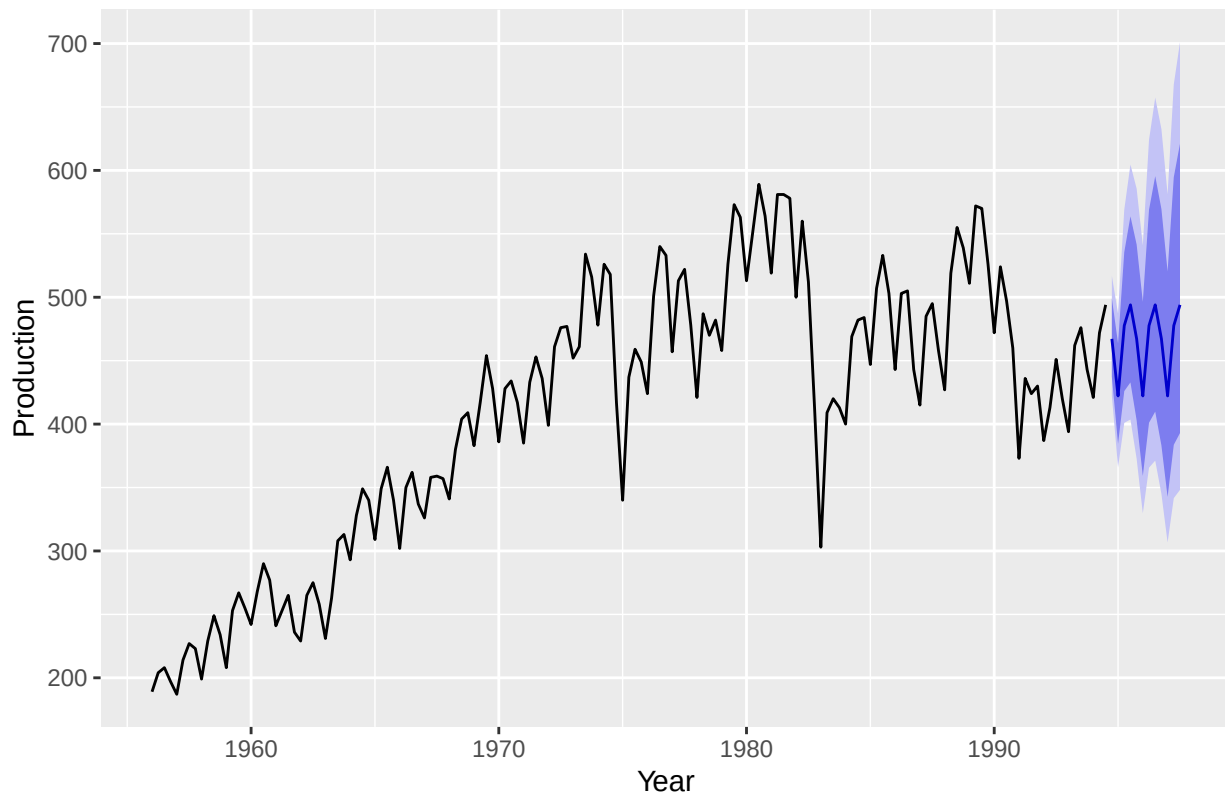
d)

```
fc_log_fix <- stlf(y1, s.window = "periodic", method = "naive", h = h)

fc_fix <- fc_log_fix
fc_fix$mean <- exp(fc_log_fix$mean)
fc_fix$lower <- exp(fc_log_fix$lower)
fc_fix$upper <- exp(fc_log_fix$upper)
fc_fix$x <- y # so plotting uses original-scale history

autoplot(fc_fix) +
  labs(title = "Bricks: stlf() reseasonalised forecast (fixed seasonality)",
       x = "Year", y = "Production")
```

Bricks: stlf() reseasonalised forecast (fixed seasonality)



e)

The errors appear to be relatively uncorrelated under the seasonalized model. There isn't a clear over/under estimation bias. In the naive forecast, the prediction is a horizontal line with a large confidence interval, but the seasonal, more advanced prediction seems to follow the same pattern as the data that preceded it.

f)

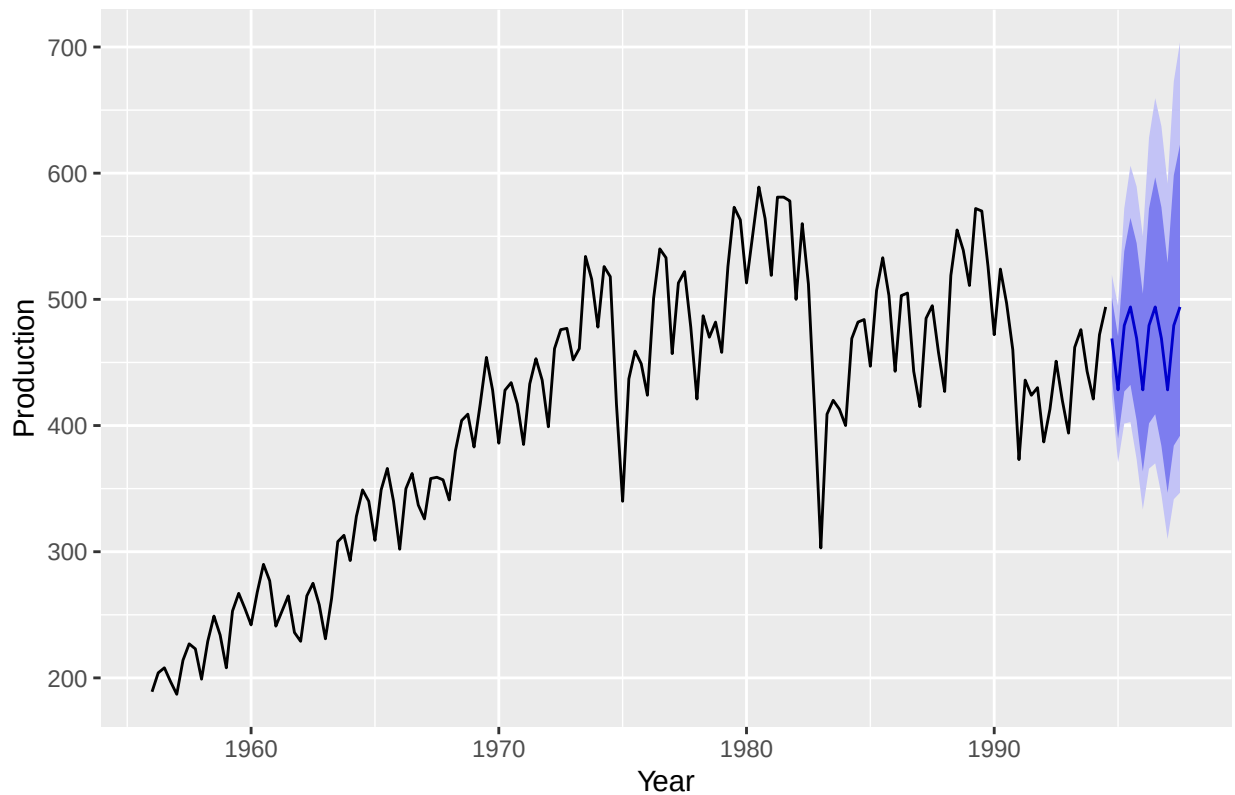
```
h <- 12

fc_log_rb <- stlf(y1, s.window = "periodic", method = "naive",
                 robust = TRUE, h = h)

fc_rb <- fc_log_rb
fc_rb$mean <- exp(fc_log_rb$mean)
fc_rb$lower <- exp(fc_log_rb$lower)
fc_rb$upper <- exp(fc_log_rb$upper)
fc_rb$x <- y

autoplot(fc_rb) +
  labs(title = "Bricks: stlf() forecast (ROBUST STL, periodic seasonality)",
       x = "Year", y = "Production")
```

Bricks: stlf() forecast (ROBUST STL, periodic seasonality)



There doesn't seem to be much of a difference between the robust stl and the stlf on seasonal data. Both have a seasonal prediction pattern with a detailed, uncorrelated confidence interval

g)

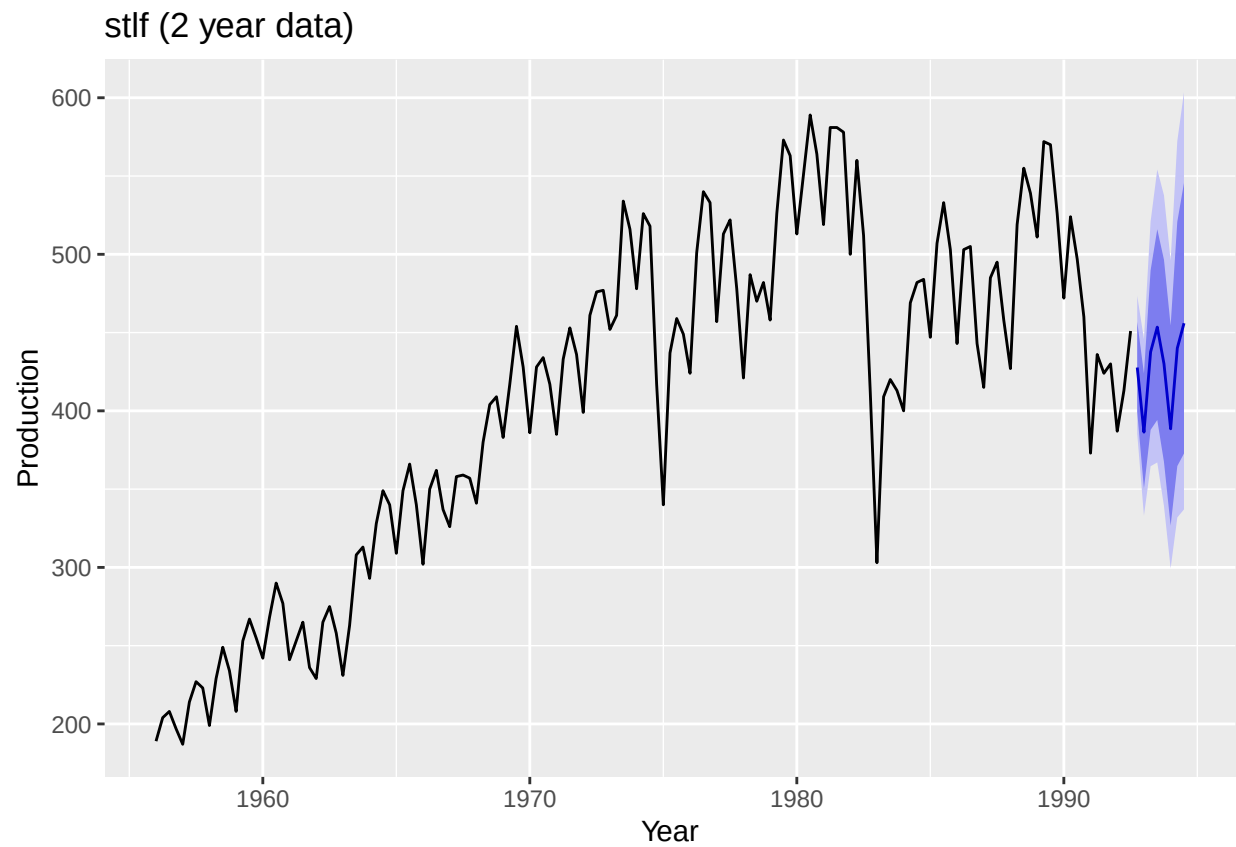
```
y <- bricksq
h <- 8

n <- length(y)
train <- ts(head(y, n - h), frequency = frequency(y), start = start(y))
test <- ts(tail(y, h), frequency = frequency(y),
           start = tsp(y)[1] + (n - h)/frequency(y))

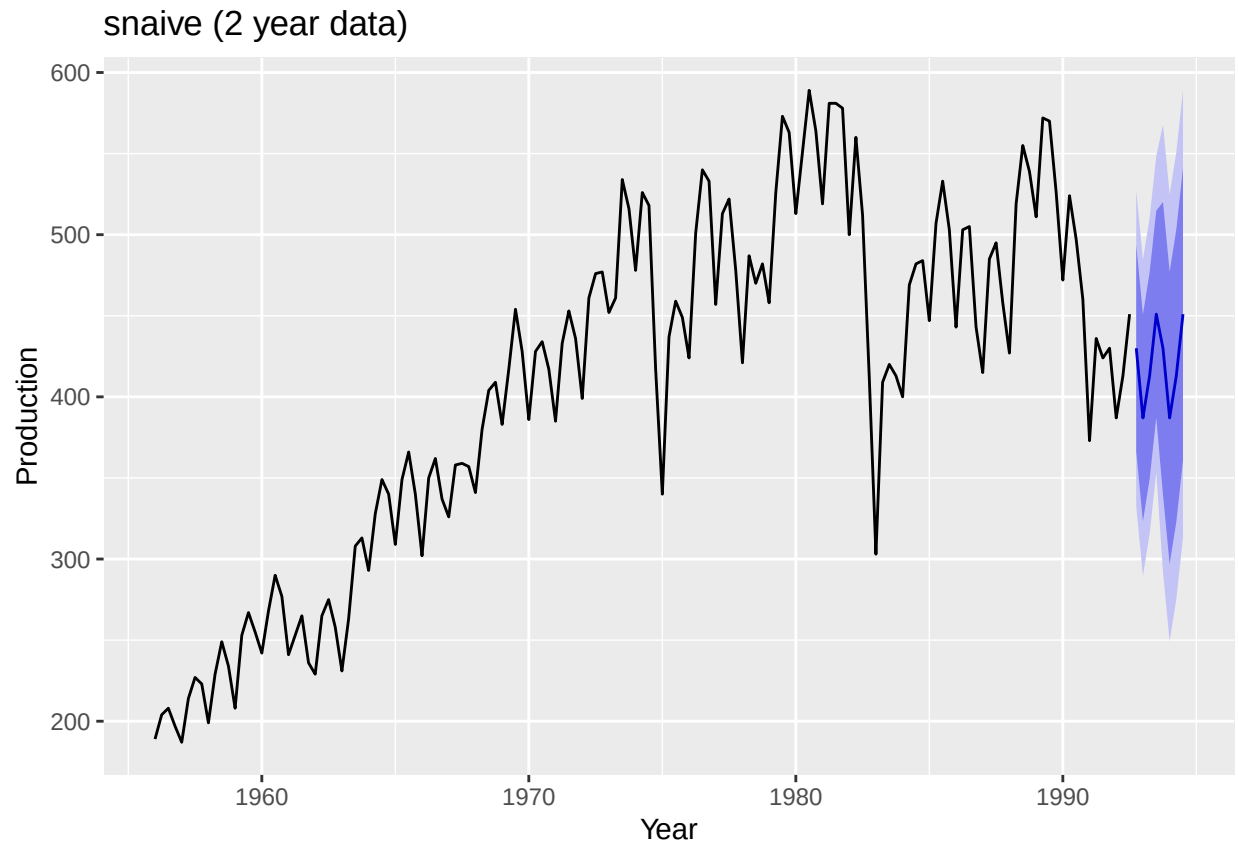
fc_stlf <- stlf(train, s.window = "periodic", method = "naive",
               lambda = 0, biasadj = TRUE, h = h)

fc_snaive <- snaive(train, h = h)

autoplot(fc_stlf) +
  labs(title = "stlf (2 year data)",
       x = "Year", y = "Production")
```



```
autoplot(fc_snaive) +  
  labs(title = "snaive (2 year data)",  
        x = "Year", y = "Production")
```



Now that we are only predicting on 2 years of data, the two methods seem to be of the same quality. Their confidence intervals are slightly different (stlf is slightly more precise), but the middle line indicating the actual forecast value is almost the same in each graph. The STL is better, but not by much.

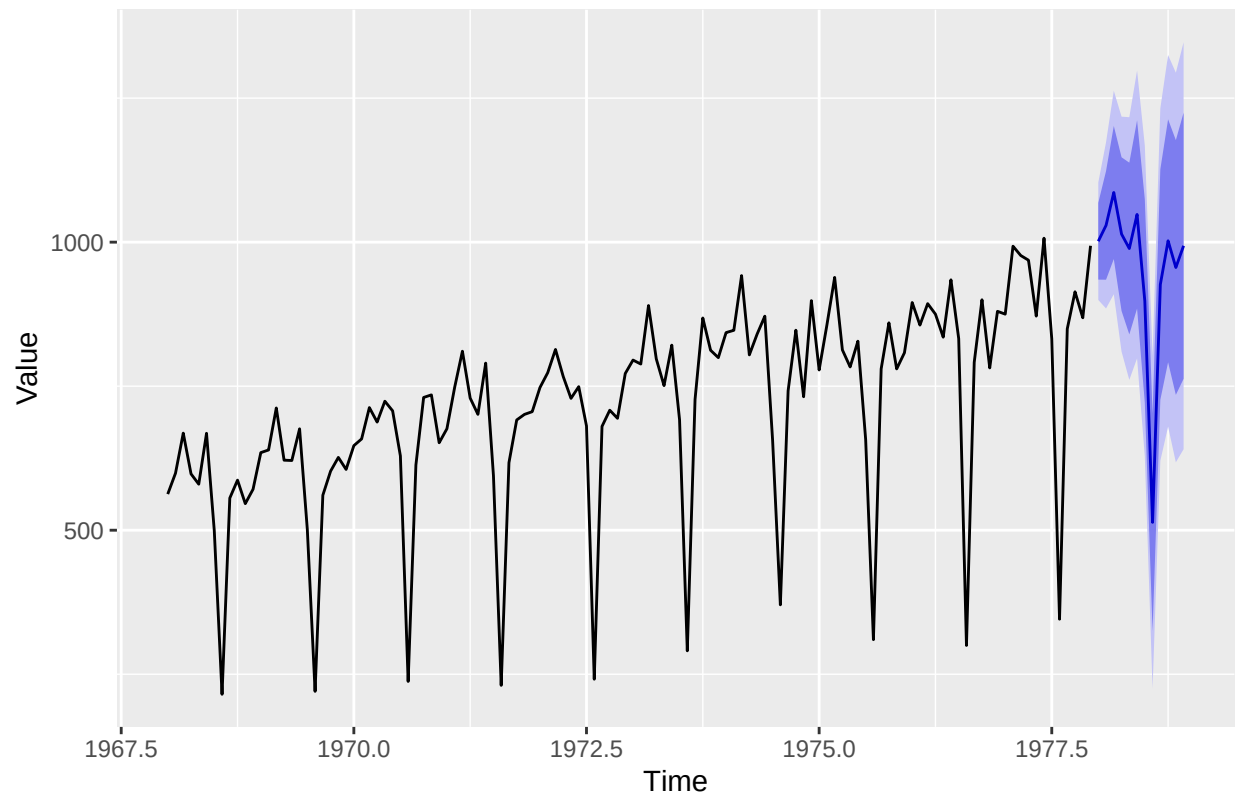
6.7

```
y <- writing
h <- 12
lam <- BoxCox.lambda(y)

fc_naive_raw <- stlf(y, s.window = "periodic", method = "naive", h = h)
fc_naive_bc <- stlf(y, s.window = "periodic", method = "naive", h = h, lambda = lam, biasadj = TRUE)
fc_rw_raw <- stlf(y, s.window = "periodic", method = "rwdrift", h = h)
fc_rw_bc <- stlf(y, s.window = "periodic", method = "rwdrift", h = h, lambda = lam, biasadj = TRUE)

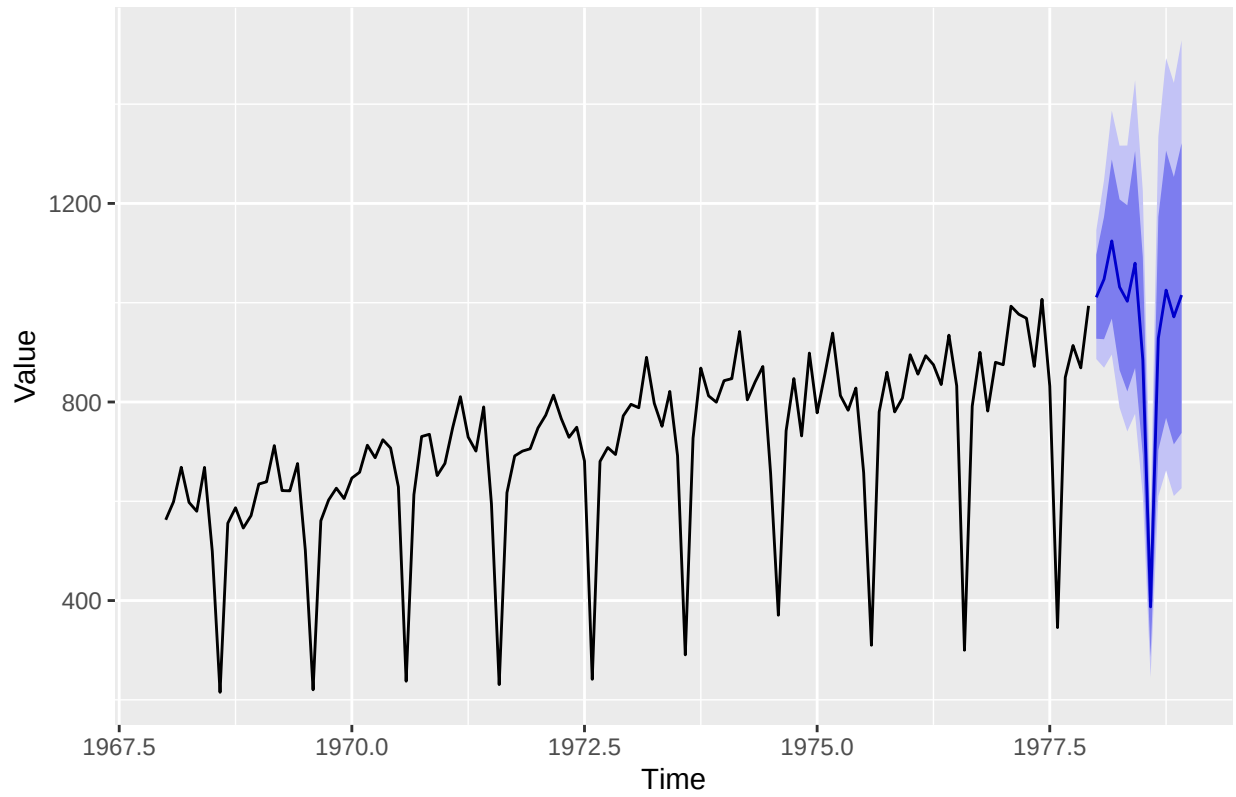
autoplot(fc_naive_raw) + labs(title = "writing: stlf (method = naive, no Box-Cox)", x = "Time", y = "Value")
```

writing: stlf (method = naive, no Box-Cox)



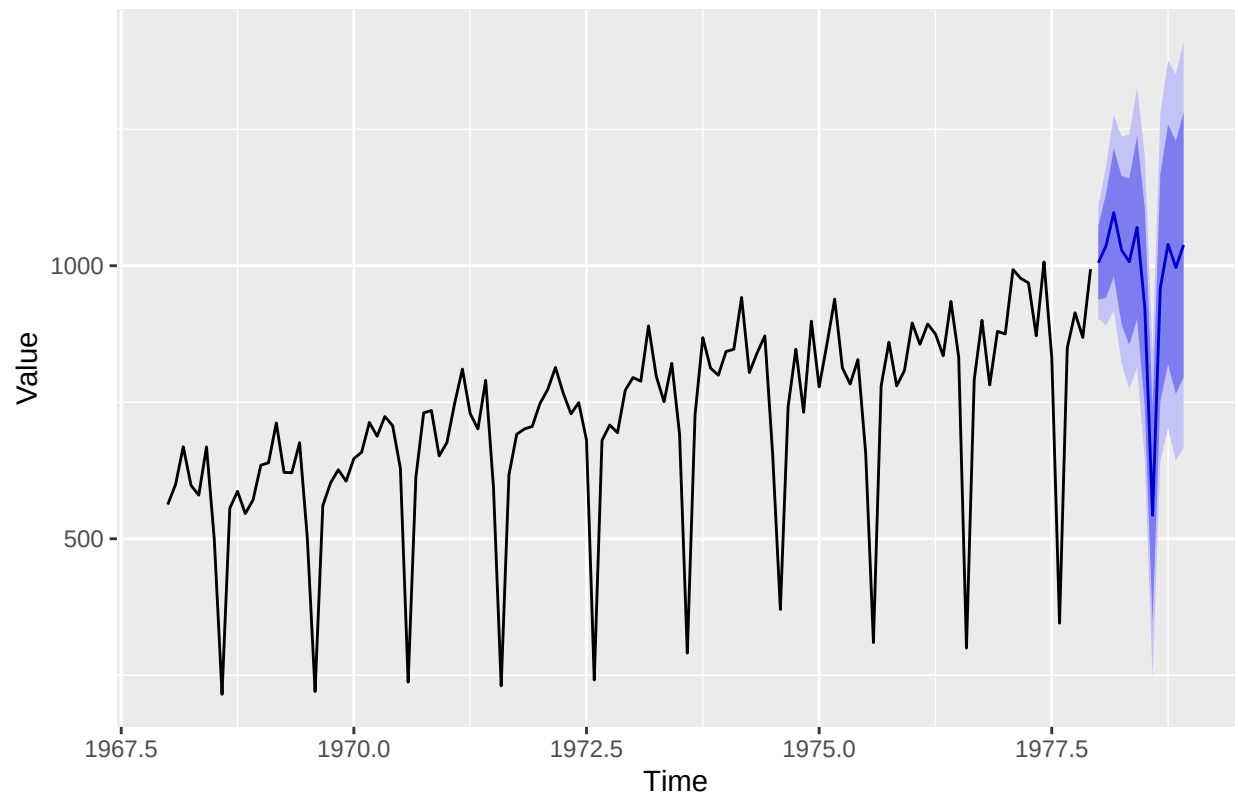
```
autoplot(fc_naive_bc) + labs(title = sprintf("writing: stlf (method = naive, Box-Cox Lambda=%.3f)", lam
```


writing: stlf (method = naive, Box-Cox Lambda=0.156)



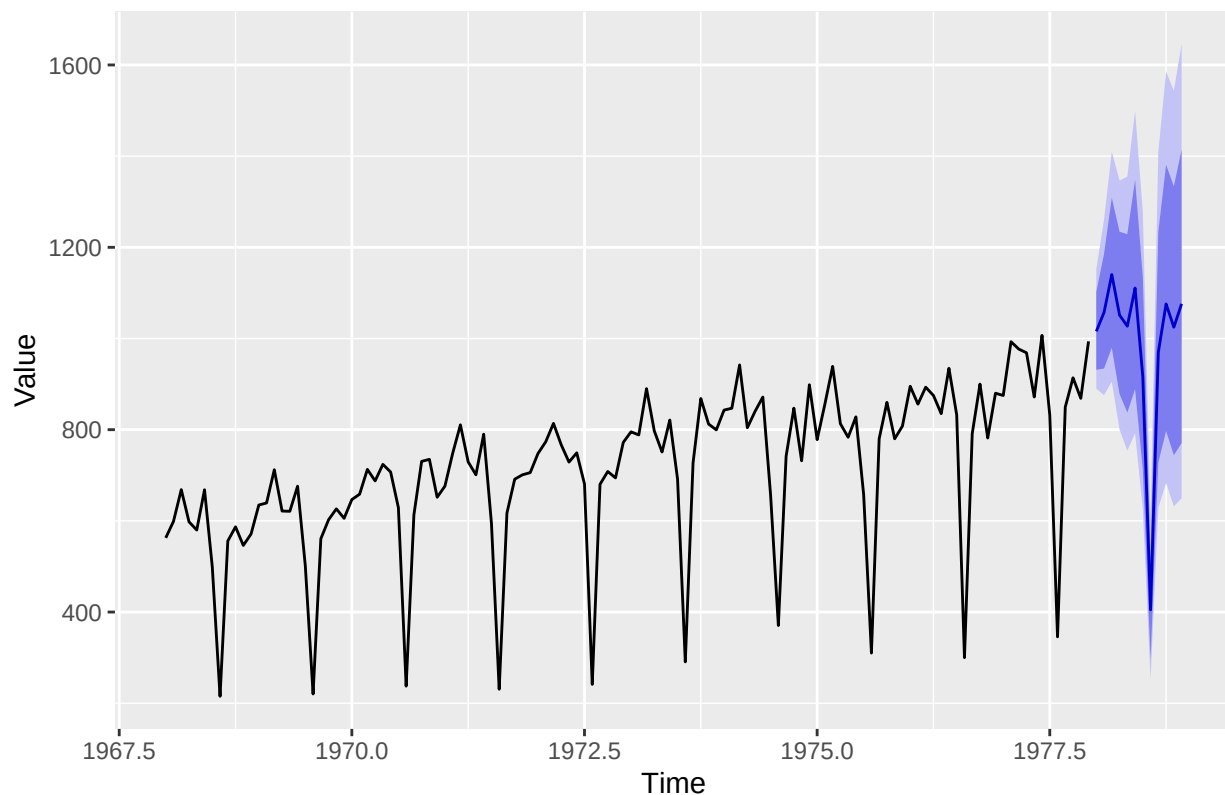
```
autoplot(fc_rw_raw) + labs(title = "writing: stlf (method = rwdrift, no Box-Cox)", x = "Time", y = "Val
```

writing: stlf (method = rwdrift, no Box-Cox)



```
autoplot(fc_rw_bc) + labs(title = sprintf("BEST: writing: stlf (method = rwdrift, Box-Cox Lambda=%.3f)"
```

BEST: writing: stlf (method = rwdrift, Box-Cox Lambda=0.156)



```
trend_needed <- ndiffs(BoxCox(y, lam)) > 0
method_used <- if (trend_needed) "rwdrift" else "naive"

trend_needed
```

```
## [1] TRUE
```

```
method_used
```

```
## [1] "rwdrift"
```

These are the four possible options, but based on the ndiffs, rwdrift is a better model. It is best to leave lambda in because it helps stabilize the prediction. All four methods look similar, but a rwdrift with lambda makes for the best long term results.

6.8

```
y <- fancy
h <- 12
lam <- BoxCox.lambda(y)

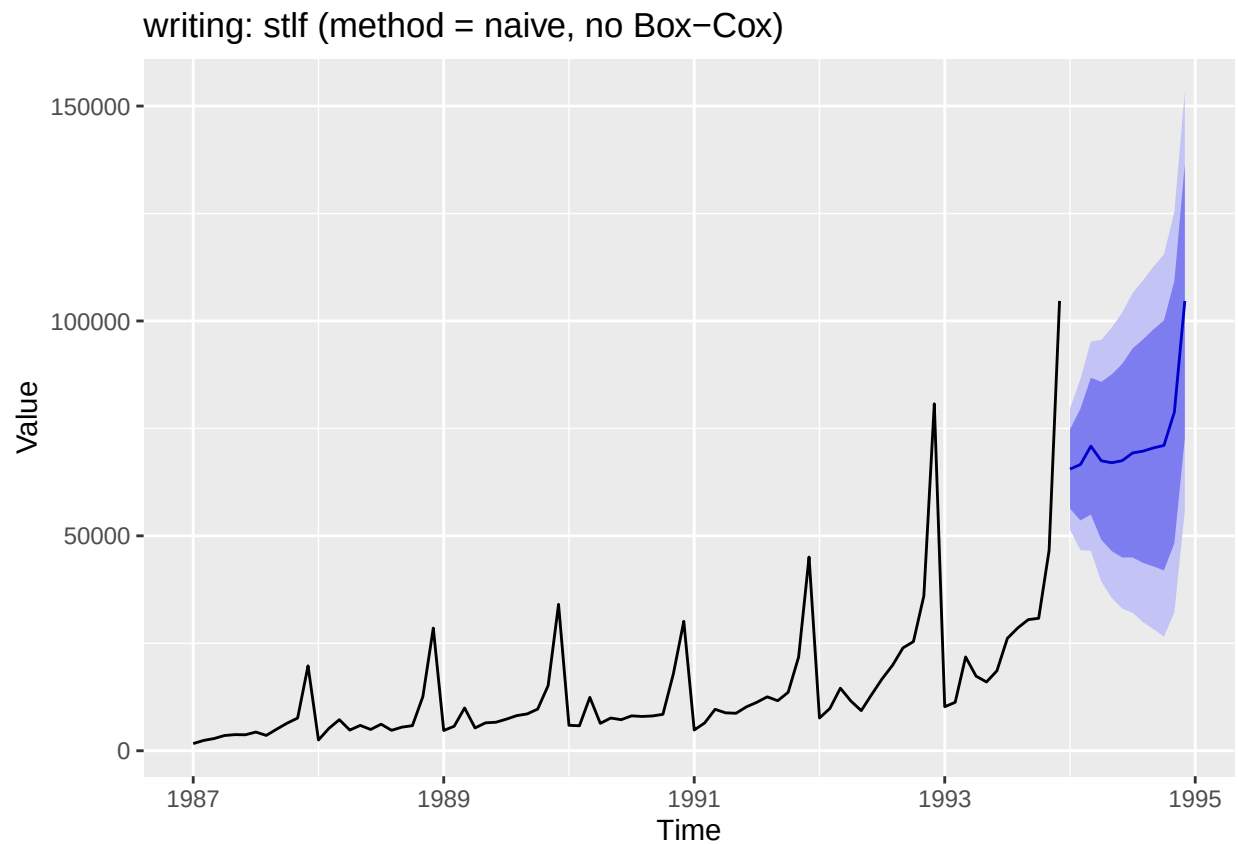
fc_naive_raw <- stlf(y, s.window = "periodic", method = "naive", h = h)
```

```

fc_naive_bc <- stlf(y, s.window = "periodic", method = "naive", h = h, lambda = lam, biasadj = TRUE)
fc_rw_raw <- stlf(y, s.window = "periodic", method = "rwdrift", h = h)
fc_rw_bc <- stlf(y, s.window = "periodic", method = "rwdrift", h = h, lambda = lam, biasadj = TRUE)

autoplot(fc_naive_raw) + labs(title = "writing: stlf (method = naive, no Box-Cox)", x = "Time", y = "Va.

```

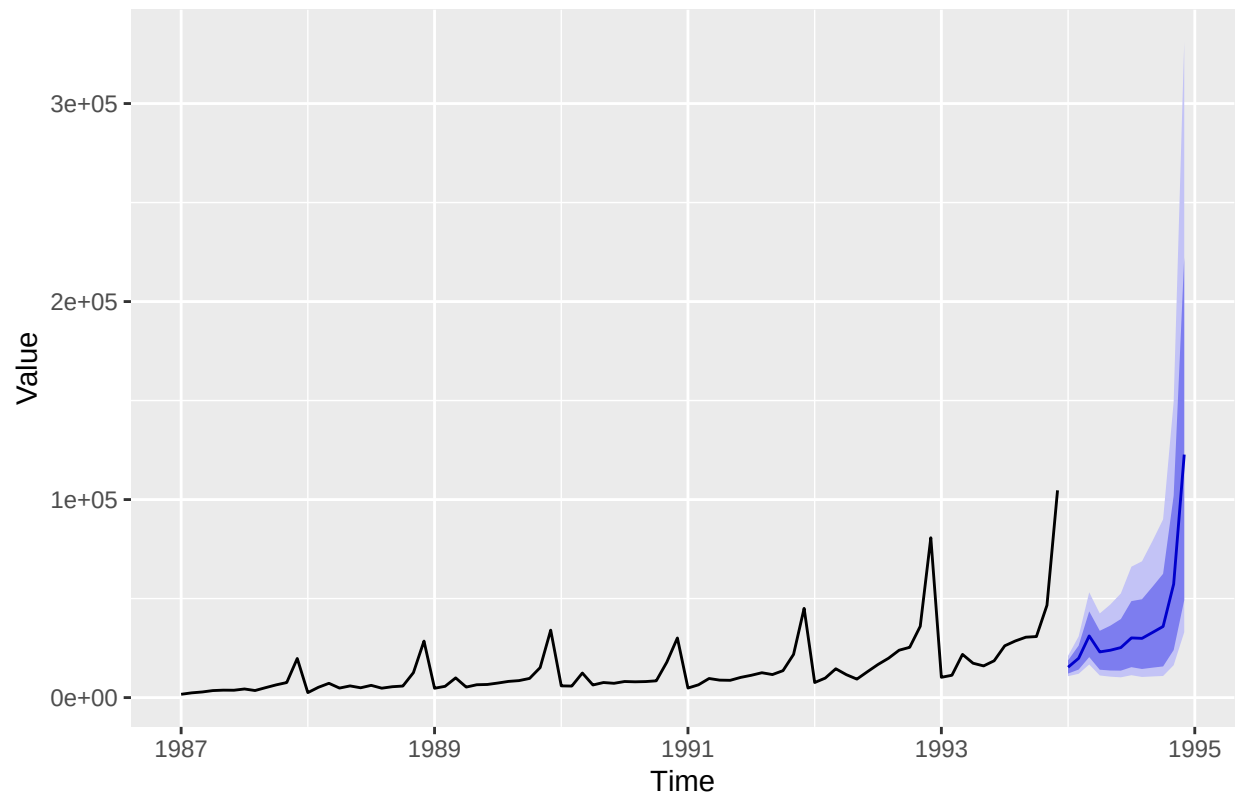


```

autoplot(fc_naive_bc) + labs(title = sprintf("writing: stlf (method = naive, Box-Cox Lambda=%.3f)", lam

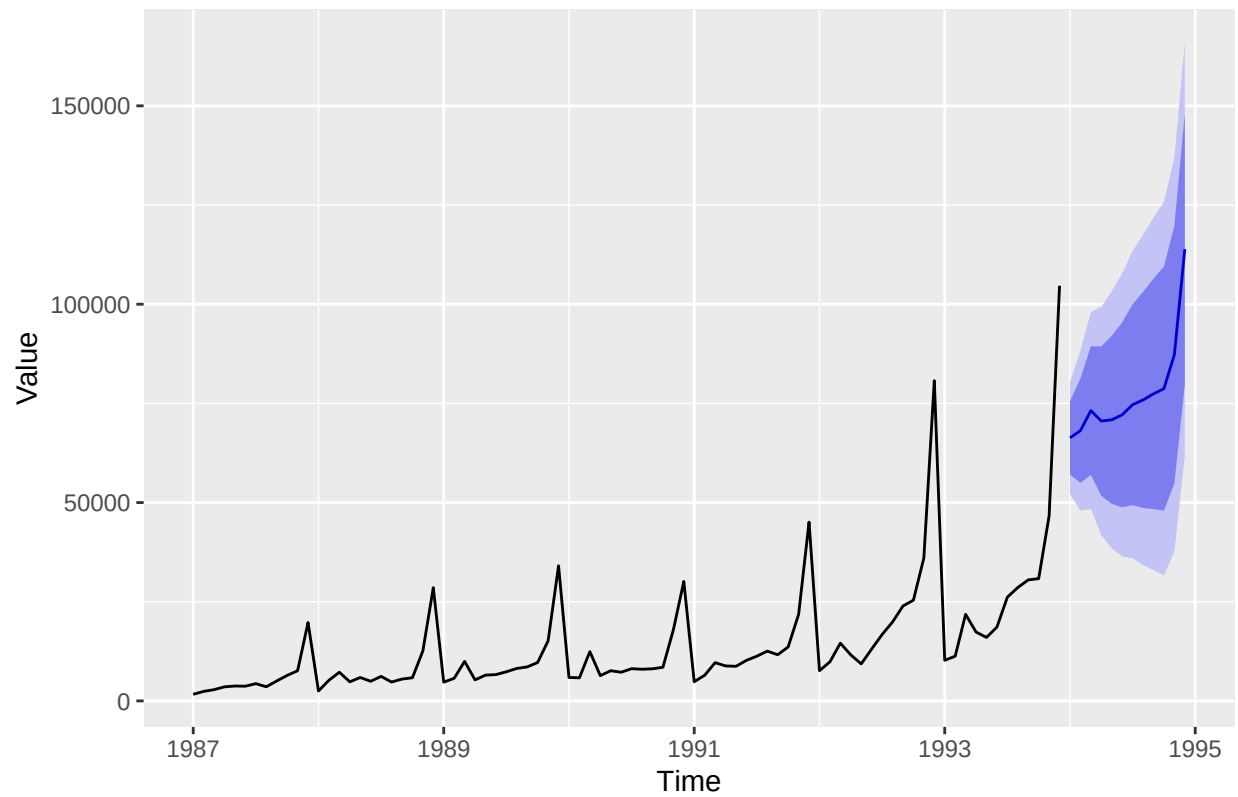
```

writing: stlf (method = naive, Box-Cox Lambda=0.002)



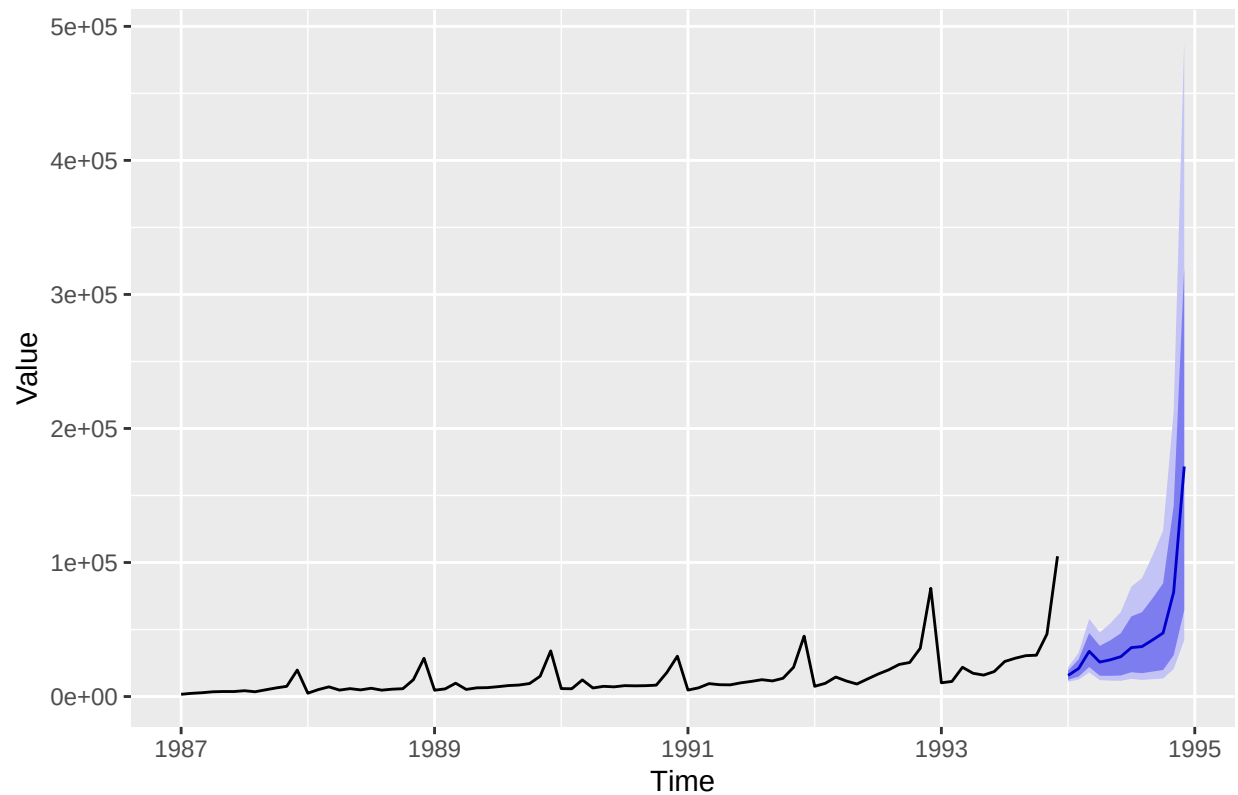
```
autoplot(fc_rw_raw) + labs(title = "writing: stlf (method = rwdrift, no Box-Cox)", x = "Time", y = "Val
```

writing: stlf (method = rwdrift, no Box-Cox)



```
autoplot(fc_rw_bc) + labs(title = sprintf("BEST: writing: stlf (method = rwdrift, Box-Cox Lambda=%.3f)"
```

BEST: writing: stlf (method = rwdrift, Box-Cox Lambda=0.002)



```
trend_needed <- ndiffs(BoxCox(y, lam)) > 0
method_used <- if (trend_needed) "rwdrift" else "naive"
```

```
trend_needed
```

```
## [1] TRUE
```

```
method_used
```

```
## [1] "rwdrift"
```

Same as in 6.7, all four models are plotted above. Again, the `ndiffs()` function proves that the `rwdrift` method is best, and the `lamda` adds in a smoother prediction. None of the combinations make for a super convincing model due to lack of movement before the key spike, but they all show steady improvement and match prior trends.